

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Ciencias Exactas e Ingenierías

División de Tecnologías para la Integración Ciber-Humana



**Captura de movimiento corporal en 3D con visión estéreo: análisis de movimiento en la
terapia física, y aplicación en Python/Unity**

Modalidad

Tesis, Tesina e Informes

Opción

Tesis

QUE PARA OBTENER EL GRADO DE

Ingeniería Informática

PRESENTA

José de Jesús Fernández García

DIRECTOR

Dr. Noé Ortega Sánchez

Guadalajara, Jalisco, septiembre de 2024

Índice general

1. Introducción	8
2. Justificación.....	11
2.1. Objetivos.....	11
2.2. Antecedentes.....	13
2.3. Impacto social.....	13
3. Metodología.....	14
4. Modelo de cámara	18
4.1. Modelo de cámara Pinhole	18
4.2. Calibración de cámara	36
4.3. Comprobación de los parámetros de la cámara	52
5. Proyección de 2D a 3D.....	57
5.1. Modelos de deep learning para la detección de puntos 2D característicos del cuerpo...	57
5.2. Estimaciones 3D de puntos característicos del cuerpo	59
5.3. Análisis del movimiento corporal en 3D	68
6. Aplicación de Python y Unity	73
6.1. Aplicación de Python.....	73
6.2. Aplicación de Unity	74
6.3. Comunicación y envío de datos entre la aplicación de Python y la aplicación de Unity	77
6.4. Interacción con modelos 3D en Unity a partir de la estimación 3D de puntos característicos del cuerpo.....	80
7. Conclusiones.....	86

Bibliografía.....	88
Anexos.....	89
1. Solución ecuación $Ax=b$	89
2. Solución ecuación $Ax=0$	90
3. Optimizador Adam	92
4. Ángulos de Euler a partir de estimaciones de puntos 3D	94

Índice de figuras

Figura 1. Tablero de ajedrez y marcador ArUco, respectivamente	15
Figura 2. Interfaz del script bash app.sh.....	17
Figura 3. Funcionamiento de una cámara Pinhole. [7]	18
Figura 4. Distorsión producida por la lente de una cámara. [8]	19
Figura 5. Ecuaciones que se pueden obtener de triángulos semejantes	20
Figura 6. Dibujo en base a la Figura 3 para la obtención de la componente en X de la proyección del punto 3D sobre el plano de imagen	22
Figura 7. Dibujo en base a la Figura 3 para la obtención de la componente en Y de la proyección del punto 3D sobre el plano de imagen	22
Figura 8. Transformaciones entre sistemas de coordenadas sobre el plano de imagen de la cámara	24
Figura 9. Relación entre el sistema de coordenadas de la cámara y el sistema de coordenadas del mundo	29

Figura 10. Dibujo en base a la Figura 3 para la obtención de la componente en X de la proyección del punto 3D sobre el plano de imagen tomando en cuenta la distorsión de la lente de la cámara	30
Figura 11. Dibujo en base a la Figura 3 para la obtención de la componente en Y de la proyección del punto 3D sobre el plano de imagen tomando en cuenta la distorsión de la lente de la cámara	31
Figura 12. Punto 2D distorsionado sobre el plano de imagen de la cámara	35
Figura 13. Obtención de pares de puntos correspondientes 2D y 3D a partir de un tablero de ajedrez.....	40
Figura 14. Parámetros requeridos en la aplicación de Python para la calibración de la cámara	50
Figura 15. Ejemplo tablero de ajedrez de dimensiones 9,6: 9 esquinas interiores de ancho y 6 esquinas interiores de largo	51
Figura 16. Calibración de cámara en la aplicación de Python.....	51
Figura 17. Cubo dibujado con realidad aumentada sobre un tablero de ajedrez	53
Figura 18. Parámetros de una cámara específica obtenidos en el proceso de calibración	54
Figura 19. Historial de pérdida de la función objetivo	54
Figura 20. Visualización de las matrices extrínsecas (Q's) obtenidas en el proceso de calibración	55
Figura 21. Comprobación de los parámetros de la cámara mediante realidad aumentada	56
Figura 22. Puntos de referencia para el modelo MediaPipe Hands. [14]	58
Figura 23. Puntos de referencia para el modelo MediaPipe Pose. [15]	59

Figura 24. Obtención de pares de puntos correspondientes 2D y 3D a partir de un marcador ArUco	61
Figura 25. Colocación del marcador ArUco al momento de calcular la matriz extrínseca (Q)	62
Figura 26. Proyección de un punto 3D sobre los planos de imagen de las cámaras	65
Figura 27. Estimaciones de puntos 3D.....	66
Figura 28. Modelos de detección de puntos 2D seleccionables	67
Figura 29. Cálculo de matrices extrínsecas	68
Figura 30. Captura de videos	69
Figura 31. Visualización de videos imagen por imagen	70
Figura 32. Visualización de ángulos articulares de una persona	71
Figura 33. Ángulo formado por 3 puntos	72
Figura 34. Vista principal de la aplicación de Python.....	74
Figura 35. Vista desde el editor de la aplicación de Unity	76
Figura 36. Arquitectura cliente-servidor entre aplicaciones	78
Figura 37. Configuraciones para la conexión entre servidor y cliente	80
Figura 38. Jerarquía de las articulaciones del modelo 3D de la mano izquierda	83
Figura 39. Nuevas articulaciones para el modelo 3D de la mano izquierda	83
Figura 40. Equivalencia de orientaciones entre los sistemas de coordenadas de la aplicación de Python y la aplicación de Unity.....	84
Figura 41. Interacción con los modelos 3D desde el ejecutable de la aplicación de Unity	84
Figura 42. Interacción con los modelos 3D desde la aplicación de Python	85

Figura 43. Algunos sistemas de coordenadas de puntos del dedo índice de la mano izquierda.....	99
---	-----------

Índice de tablas

Tabla 1. Selección de puntos para la elaboración de sistemas de coordenadas para los diferentes modelos de deep learning basado en la estrategia planteada	99
---	-----------

1. Introducción

Hoy en día el uso de la tecnología se ha vuelto de mucha ayuda en áreas como la educación, la robótica, el entretenimiento, entre muchas otras. Pero también hay otras como la terapia física en la salud en donde no existe mucho el uso de la tecnología para acelerar algunas tareas, o para ayudar en los procesos de rehabilitación de los pacientes. Pocos fisioterapeutas son los que usan algún sistema informático con el único objetivo de llevar un control de los pacientes, y solo instituciones se pueden permitir adquirir sistemas robóticos para ayudar en los procesos de rehabilitación de los pacientes, siendo en muchos casos muy específicos y de alto costo.

Pensar en que muchas tareas en la terapia física requieren la atención del fisioterapeuta sobre el paciente en la realización de sus ejercicios, da mucho que decir sobre la importancia de la observación hacia el paciente. De aquí nace la idea de lo valioso que podría ser un sistema de visión estéreo integrado en la terapia física para tener un control y análisis del movimiento corporal del paciente.

La visión estéreo es una técnica de la visión artificial que funciona similar a la percepción 3D en la visión humana. Muchas de sus aplicaciones están destinadas a la robótica en la parte de la percepción de los robots, y es muy poco usada en otras áreas que no tengan que ver con la robótica.

Con la visión estéreo se busca estimar la pose en 3D de una persona para realizar análisis del movimiento corporal.

La diferencia entre los métodos tradicionales para la estimación de la pose que utilizan una sola cámara y la visión estéreo que utiliza dos, es que la visión estéreo permite tener una representación 3D de la escena capturada por las cámaras, y con una sola cámara solo es posible trabajar en un plano 2D, perdiendo noción de profundidad e información de la escena

presente. Además la visión estéreo se puede mejorar si se incrementa el número de cámaras, ya que esto ayuda a agregar más información y precisión de lo que ocurre en escena.

Existen algoritmos de deep learning [1,2,3] que trabajan con una sola cámara para entregar una representación en 3D de alguna parte del cuerpo. Estos intentan imitar lo que hace la visión estéreo, lo cual ayuda a reducir el número de cámaras, sin embargo, no suelen tener una representación 3D global de la escena y sufren problemas cuando se presentan oclusiones de partes del cuerpo.

La implementación desde cero de la visión estéreo muy difícilmente se encuentra y la que existe por lo general es difícil de entender y de llevar a cabo. La librería OpenCV [4] ofrece documentación al respecto y ofrece métodos para llevar a cabo tareas de visión estéreo, sin embargo, se pierde mucho la idea de cómo funciona realmente y de cómo esta puede aplicarse a una gran cantidad de problemas, motivo por el cual se decidió implementar desde cero la visión estéreo, lo que conlleva a un extenso trabajo y desarrollo.

La principal motivación del uso de la visión estéreo radica en la gran cantidad de aplicaciones que se pueden desarrollar en diferentes áreas:

- Salud, en especial la terapia física
 - Análisis del movimiento, biomecánica, la cual combinada con sensores podría ser más útil y poderosa
 - Juegos interactivos para la rehabilitación física (cualquier juego controlado por los movimientos del paciente)
- Robótica
 - Hacer mediciones del entorno para la percepción del robot
 - Mover robots por control de movimiento
- Entretenimiento
 - Captura de movimiento

- ◆ Películas
- ◆ Videojuegos
- Otros
 - Realidad aumentada
 - Reconstrucción 3D
 - Eliminar distorsión de imágenes
 - Creación de mapas de disparidad

Hay un sin fin de aplicaciones que se pueden desarrollar entendiendo la visión estéreo y los conceptos fundamentales de la visión artificial.

El presente trabajo busca crear un sistema informático que sirva como herramienta en la terapia física, que sea económico, práctico, y que sea un primer enfoque a las posibilidades que se pueden alcanzar con el uso de la visión estéreo, tal como lo es el análisis del movimiento corporal en 3D y su posible uso para la rehabilitación física mediante juegos interactivos 3D creados en Unity que responden al movimiento del paciente.

2. Justificación

La principal razón por la cual se pretende desarrollar un sistema de visión estéreo en la terapia física es para que sea una herramienta que favorezca y ayude a los fisioterapeutas durante el proceso de rehabilitación de los pacientes, ofreciendo soluciones y alternativas a tareas ya existentes para que se lleven de una mejor manera y posiblemente más efectiva, tal como es el caso del análisis del movimiento en 3D y de la posible integración con Unity para desarrollar juegos interactivos 3D para la rehabilitación física del paciente.

Se busca que sea un primer enfoque a todas las posibles aplicaciones que podría tener dentro del área de la salud, siempre con el objetivo de ayudar al fisioterapeuta y de hacer más amena la rehabilitación del paciente en el caso de que se lleve a cabo mediante juegos interactivos 3D en Unity controlados por el movimiento corporal del paciente.

Además, dada la idea de que un sistema informático por lo general no se adquiere debido a su alto costo, el sistema que se pretende desarrollar es económico y fácil de usar, ya que solo necesita de una computadora con sistema operativo Linux donde pueda correr el sistema y de dos cámaras USB de bajo costo.

2.1. Objetivos

Como objetivo general se busca innovar con un sistema informático de bajo costo que implementa visión estéreo mediante dos cámaras USB, de tal forma que sirva como herramienta en el área de la terapia física para tareas de análisis del movimiento corporal en 3D por medio de video, y que además sea una motivación y un primer enfoque a las posibilidades que se pueden alcanzar con su uso en la rehabilitación física mediante juegos interactivos 3D creados en Unity que responden al movimiento corporal del paciente.

Áreas de participación:

- Desarrollo tecnológico

Retos:

- Tecnologías de la Información (TIC)

- Salud

Retos:

- Medicina preventiva y atención de la salud

Objetivos específicos:

- Crear un sistema informático económico que implemente visión estéreo, que sea capaz de usarse en una clínica de terapia física y que sea fácil de instalar.
- Implementar estimaciones 3D de partes características del cuerpo humano en tiempo real.
- Analizar el movimiento corporal en 3D a partir de videos pregrabados.
- Crear un primer enfoque a las posibilidades que se pueden alcanzar con el uso de la visión estéreo, así como su posible uso para la rehabilitación física mediante juegos interactivos 3D creados en Unity que responden al movimiento corporal del paciente.
- Conseguir que el sistema informático sea una herramienta que favorezca y ayude a los fisioterapeutas, ofreciendo soluciones y alternativas a tareas ya existentes para que se lleven de una mejor manera y posiblemente más efectiva.

2.2. Antecedentes

En la terapia física, los fisioterapeutas realizan muchas pruebas de evaluación a sus pacientes para determinar con exactitud los posibles tratamientos que mejor se acomodan al paciente, y una de esas pruebas es el análisis del movimiento, la cual sirve para determinar posibles alteraciones en la postura del paciente. Posterior a la evaluación del paciente, la rehabilitación física juega un papel muy importante en la mejora de la salud del paciente, la cual consiste en ejercicios específicos que se deben seguir a lo largo del tratamiento.

La implementación de la visión estéreo en sistemas informáticos no es tan común en áreas como la salud, y muchos de los motivos se deben a que se desconocen las aplicaciones y además no es tan fácil desarrollar un sistema de visión estéreo desde cero, esto por su complejidad.

Un trabajo relacionado al análisis del movimiento corporal en video es Kinovea [5], el cual es un software de escritorio muy enfocado a la actividad deportiva y no usa visión estéreo. Es gratuito y ofrece soluciones muy buenas, tal como mediciones articulares, seguimiento de puntos a lo largo del tiempo, entre otros. Todas las soluciones trabajan sobre un video y la información que este produce a lo largo del tiempo, sin embargo, se suele perder la noción de profundidad, de un sistema de referencia 3D, y de otras cosas más. Con la visión estéreo se pueden mitigar esos problemas y se puede dar más información con respecto a la escena presente.

2.3. Impacto social

El análisis del movimiento corporal en 3D será más eficiente por medio de la visión estéreo, a tal grado de tener aplicaciones en Unity para la rehabilitación física de los pacientes mediante juegos interactivos 3D creados en Unity que responden al movimiento del paciente.

3. Metodología

El desarrollo del sistema se llevó a cabo siguiendo la metodología de desarrollo en espiral y consiste en una aplicación de escritorio creada con Python utilizando el framework CustomTkinter. La aplicación se encuentra encapsulada con Docker para su fácil distribución.

Además de la aplicación de Python también se creó un ejecutable en Unity en donde se capturan datos enviados por Socket TCP/IP desde la aplicación Python para mover modelos 3D a partir de la captura del movimiento corporal de una persona. Se optó por una arquitectura cliente-servidor para la comunicación entre las aplicaciones, donde la aplicación de Unity actúa como cliente y la aplicación de Python actúa como servidor.

El sistema es específico para sistemas operativos Linux que cuenten con Docker y Docker-Compose, esto debido a que la aplicación Python se encuentra encapsulada con Docker y el ejecutable de Unity está creado para ejecutarse en sistemas Linux. Se usa Docker-Compose para lanzar la aplicación Python como un servicio y para configurar dicha aplicación.

De los aspectos más importantes a destacar del sistema de visión estereó son el uso del modelo de cámara Pinhole (el cual es un modelo matemático del cual se fundamenta la visión estereó) y de modelos de deep learning para la estimación de puntos 2D característicos del cuerpo sobre una imagen.

Los materiales que fueron utilizados son los siguientes:

- PC de escritorio o laptop
- 2 cámaras USB con resolución de video 480p
- 2 trípodes (opcional, la intención es que las cámaras estén fijas y puedan manipularse fácilmente)

- 2 patrones conocidos
 - Tablero de ajedrez
 - Marcador ArUco

Con respecto a los patrones conocidos, es importante que sean impresos y pegados en alguna tabla rígida para su posterior uso. Pueden estar pegados en la misma tabla por lados diferentes. Los tipos y las medidas de dichos patrones serán muy importantes más adelante, por lo que se debe tomar en cuenta. Además, estos patrones son de suma importancia para varias tareas y pueden existir de muchos tipos y dimensiones. Los patrones que se usaron se muestran en la Figura 1 y son usados en los siguientes casos:

- Tablero de ajedrez: Es usado en el proceso de calibración de la cámara
- Marcador ArUco: Es usado en el proceso previo a la estimación de puntos 3D

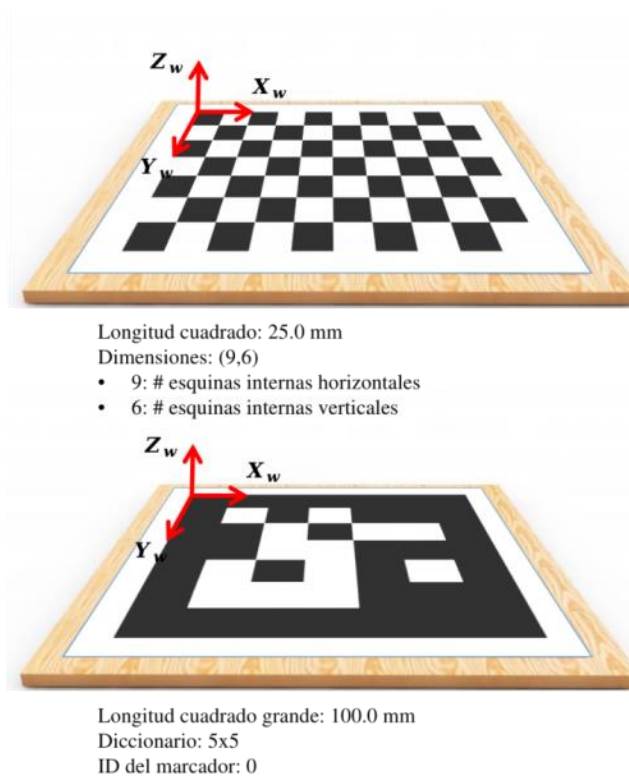


Figura 1. Tablero de ajedrez y marcador ArUco, respectivamente.

El sistema de coordenadas que se encuentra definido en la Figura 1 es el sistema de coordenadas del mundo (w), donde primeramente están definidos los puntos 3D. Las unidades que se utilizan son los milímetros.

Todo el sistema se encuentra dividido en dos aplicaciones, una de Python y otra de Unity, las cuales están distribuidas de la siguiente manera:

- app.sh (script bash)

Este script permite automatizar el proceso de lanzamiento del servicio de la aplicación de Python. La interfaz de línea de comandos del script se muestra en la Figura 2.

- app (carpeta)

Aquí se encuentra todo el código de la aplicación de Python. Además se encuentra el entorno virtual de Python.

- app_docker (carpeta)

- docker-compose.yaml

Archivo de Docker-Compose para el lanzamiento de la aplicación de Python.

- Dockerfile

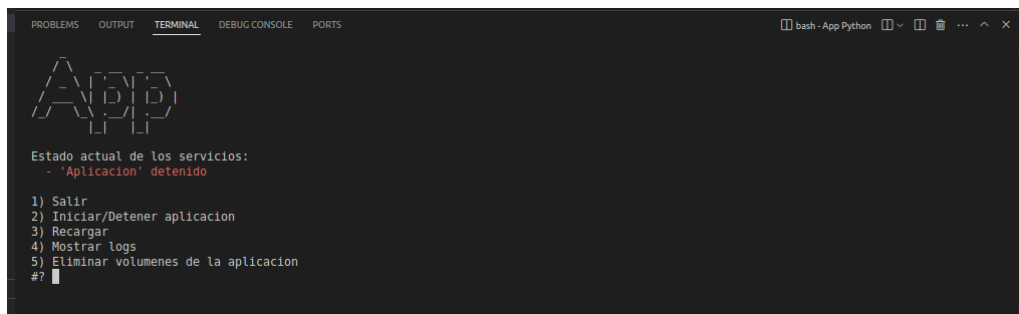
Archivo para la construcción de la imagen de Docker de la aplicación de Python.

- requirements.txt

Archivo de paquetes necesarios utilizados en la aplicación de Python (archivo necesario durante la construcción de la imagen de Docker). Este archivo se obtiene a partir de los paquetes instalados en el entorno virtual de Python.

- unity (carpeta)

Aquí se encuentra el ejecutable de Unity, así como el proyecto completo en Unity para modificaciones futuras.



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
bash - App Python

  A  E  D
 _A_ E _D_
|_A_|_|E_|_|D_|
|_|_|_|_|_|_|_|_|

Estado actual de los servicios:
- 'Aplicacion' detenido

1) Salir
2) Iniciar/Detener aplicacion
3) Recargar
4) Mostrar logs
5) Eliminar volúmenes de la aplicacion
#? 
```

Figura 2. Interfaz del script bash app.sh.

El desarrollo de la aplicación de Python fue en un entorno virtual de Python en local y al término del desarrollo se creó la imagen de Docker para encapsular toda la aplicación. El desarrollo de la aplicación de Unity se llevó a cabo en local.

Versiones de las tecnologías utilizadas:

- Python: 3.8.10
- Docker: 24.0.7
- Docker-Compose: 1.25.0
- Unity
 - Unity Hub: 3.8.0
 - Editor: 2020.3.7f1

4. Modelo de cámara

En este capítulo se detallarán diversos temas relacionados con el sistema de visión estéreo. Se hablará de los fundamentos de la visión estéreo, de la calibración de cámara y de la comprobación del buen funcionamiento de los parámetros de la cámara.

4.1. Modelo de cámara Pinhole

El modelo de cámara Pinhole [6] (o estenopeica) es un modelo matemático que describe la relación matemática de un punto 3D y su proyección en el plano de imagen 2D de la cámara.

A menudo este modelo se puede utilizar como una descripción razonable de como una cámara representa una escena 3D, siempre y cuando no se usen cámaras muy complejas o que produzcan mucha distorsión.

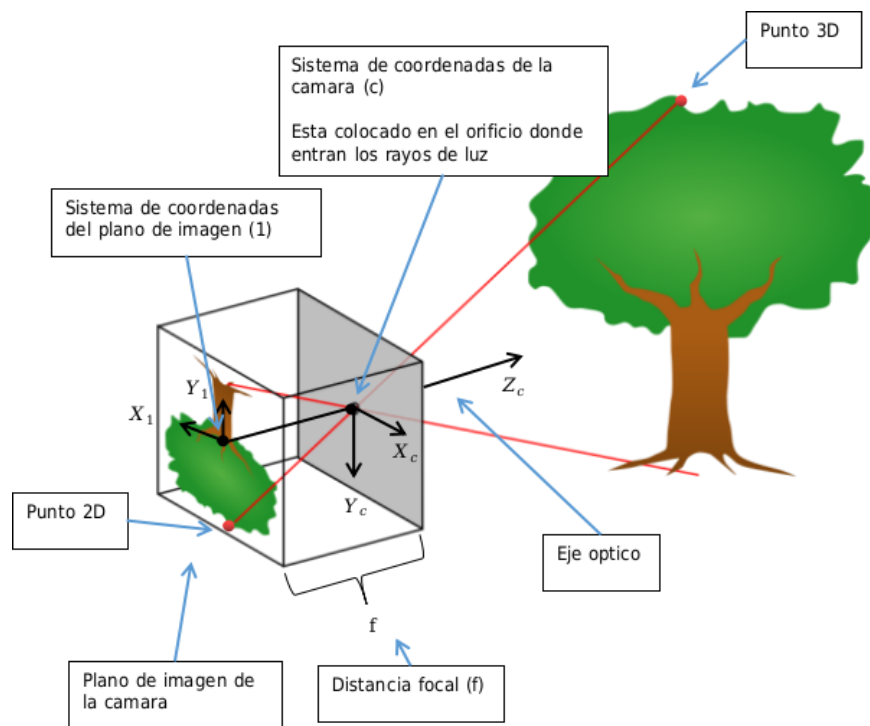


Figura 3. Funcionamiento de una cámara Pinhole. [7]

En la Figura 3, el punto donde convergen todos los rayos es donde se define el sistema de coordenadas de la cámara (sistema tridimensional). El eje óptico es normal al plano de imagen de la cámara.

El modelo de cámara Pinhole se puede dividir en dos partes:

- Modelo ideal: No toma en cuenta las distorsiones que produce la lente de la cámara
- Modelo no ideal: Toma en cuenta las distorsiones que produce la lente de la cámara
 - Tipos de distorsiones más comunes:
 - ◆ Radial
 - ◆ Tangencial

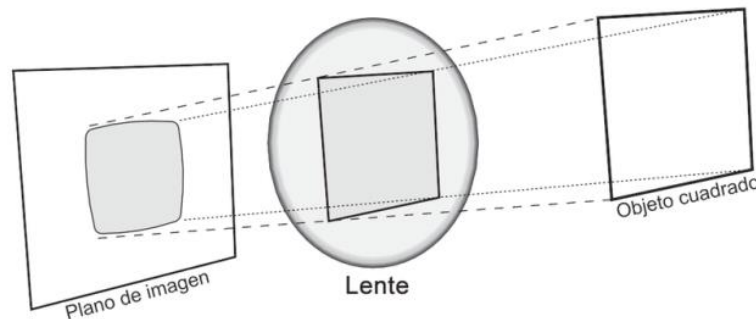


Figura 4. Distorsión producida por la lente de una cámara. [8]

El tema de la distorsión producida por la lente se explica a detalle en [9,10].

El modelo como tal está basado en los resultados que produce la Figura 3. Estos resultados corresponden al modelo de cámara Pinhole ideal. Para el modelo de cámara Pinhole no ideal los resultados son algo diferentes, pero tienen un análisis similar, la única diferencia es que toma en cuenta la refracción que tienen los rayos de luz al pasar por la lente de la cámara, tal y como se muestra en la Figura 4.

En la práctica realmente se utilizan los dos modelos, el ideal y el no ideal. Mas adelante se explicará en detalle cuando se usa uno y cuando se usa otro.

Antes de continuar es importante entender algunos conceptos:

- La distancia focal indica el ángulo de visión, es decir, cuanto se captura de la escena.
Por ejemplo:
 - Una distancia focal de 18 mm captura más de la escena.
 - Una distancia focal de 300 mm captura menos de la escena.
- Cuando se toma una foto con una cámara, la escena 3D se proyecta al sensor de la cámara en 2D. Este mapeo se llama proyección en perspectiva.
 - Para un objeto, entre más lejos este de la cámara, más pequeño se verá y entre más cerca este de la cámara, más grande se verá.
- La distorsión producida por la lente de una cámara es causada por la refracción de la luz en el vidrio de la lente. La refracción varia a lo largo de la lente y es máxima en los bordes.
- Dos triángulos son semejantes si tienen los mismos ángulos. La Figura 5 muestra la relación que existe entre dos triángulos semejantes.

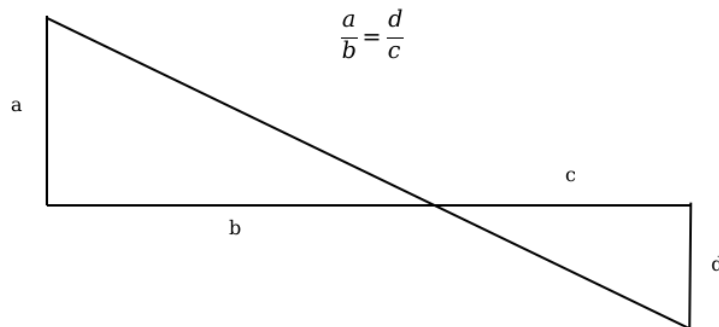


Figura 5. Ecuaciones que se pueden obtener de triángulos semejantes.

Es importante mencionar que a lo largo de todo este análisis se estarán definiendo sistemas de coordenadas en 2D y en 3D, se estarán obteniendo ecuaciones escalares y vectoriales, y se estarán haciendo conversiones entre sistemas de coordenadas. Además se hará uso de notación por bloques en expresiones vectoriales y se usaran las coordenadas homogéneas y matrices de

transformación homogénea para expresar resultados de una forma más elegante. Todo se estará detallando paso a paso.

La notación vectorial y matricial que se estará siguiendo es la siguiente:

- v_k : Punto v visto desde el sistema de coordenadas k
- t_{ij} : Vector de traslacion del sistema de coordenadas i con respecto al sistema de coordenadas j
- T_{ij} : Matriz de rotacion/transformacion del sistema de coordenadas i con respecto al sistema de coordenadas j
- i_a : Vector unitario en direccion del eje X del sistema de coordenadas a
- j_a : Vector unitario en direccion del eje Y del sistema de coordenadas a
- k_a : Vector unitario en direccion del eje Z del sistema de coordenadas a

Ahora bien, en base a la Figura 3 y viendo la imagen de diferentes perspectivas, se tienen las siguientes ecuaciones:

$v_{1x} = f \frac{v_{cx}}{v_{cz}}$	(1)
$v_{1y} = f \frac{v_{cy}}{v_{cz}}$	(2)
$v_{1nx} = \frac{v_{cx}}{v_{cz}}$	(3)
$v_{1ny} = \frac{v_{cy}}{v_{cz}}$	(4)
$v_1 = f v_{1n}$	(5)

En la Figura 6 y 7 se muestra más a detalle de donde se obtuvieron las ecuaciones anteriores. Esto gracias a los triángulos semejantes que se forman.

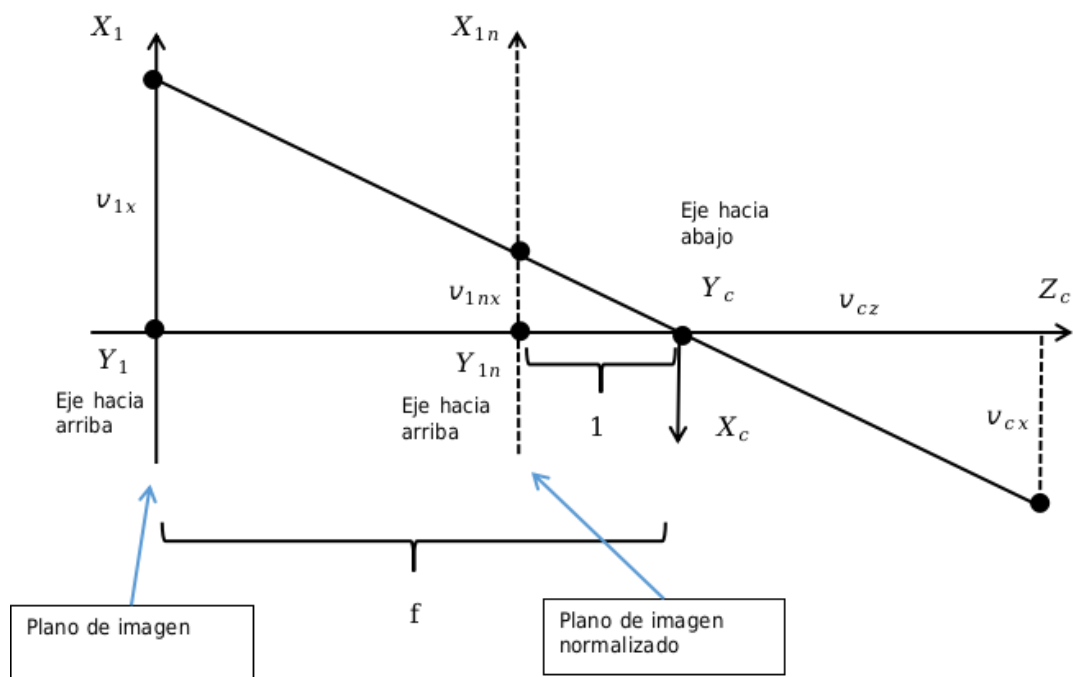


Figura 6. Dibujo en base a la Figura 3 para la obtención de la componente en X de la proyección del punto 3D sobre el plano de imagen.

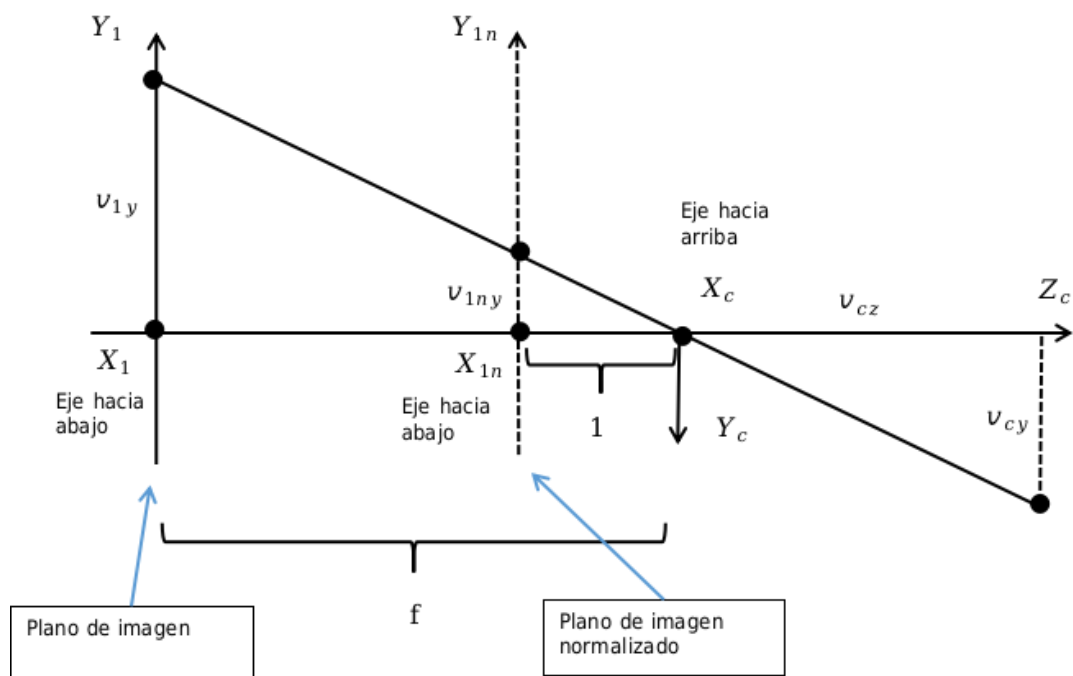


Figura 7. Dibujo en base a la Figura 3 para la obtención de la componente en Y de la proyección del punto 3D sobre el plano de imagen.

A continuación se explican los resultados:

- Punto expresado en el sistema de coordenadas 1 (sobre el plano de imagen):

$$v_1 = \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix} = \begin{bmatrix} f \frac{v_{cx}}{v_{cz}} \\ f \frac{v_{cy}}{v_{cz}} \end{bmatrix} \quad (6)$$

- Punto expresado en el sistema de coordenadas 1 normalizado (el cual se definió paralelo al plano de imagen, como se muestra en la Figura 6 y Figura 7):

$$v_{1n} = \begin{bmatrix} v_{1nx} \\ v_{1ny} \end{bmatrix} = \begin{bmatrix} \frac{v_{cx}}{v_{cz}} \\ \frac{v_{cy}}{v_{cz}} \end{bmatrix} \quad (7)$$

- Punto expresado en el sistema de coordenadas de la cámara

$$v_c = \begin{bmatrix} v_{cx} \\ v_{cy} \\ v_{cz} \end{bmatrix} \quad (8)$$

Del análisis anterior, La equ. 5 es muy importante porque nos dice lo siguiente:

$$\text{Punto plano de imagen} = (f)(\text{Punto plano de imagen normalizado})$$

La equ. 1 y 2 se pueden escribir de forma matricial de las siguientes maneras:

$$v_{cz} \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix} = \begin{bmatrix} f & 0 \\ 0 & f \end{bmatrix} \begin{bmatrix} v_{cx} \\ v_{cy} \end{bmatrix} \quad (9)$$

$$v_{cz} \begin{bmatrix} v_{1x} \\ v_{1y} \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{cx} \\ v_{cy} \\ v_{cz} \end{bmatrix} \quad (10)$$

$$v_{cz} \begin{bmatrix} v_1 \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} v_c \quad (11)$$

Cabe destacar que las unidades con las que se está trabajando hasta el momento son los milímetros (mm). Mas adelante se hará una conversión de milímetros a pixeles para los puntos sobre el plano de imagen y una conversión de milímetros a metros para los puntos en 3D.

Ahora bien, nos interesa trabajar con un sistema de coordenadas diferente sobre el plano de imagen (no con el sistema de coordenadas 1), uno que este más adecuado como lo es el sistema de coordenadas 3 que se muestra en la Figura 8.

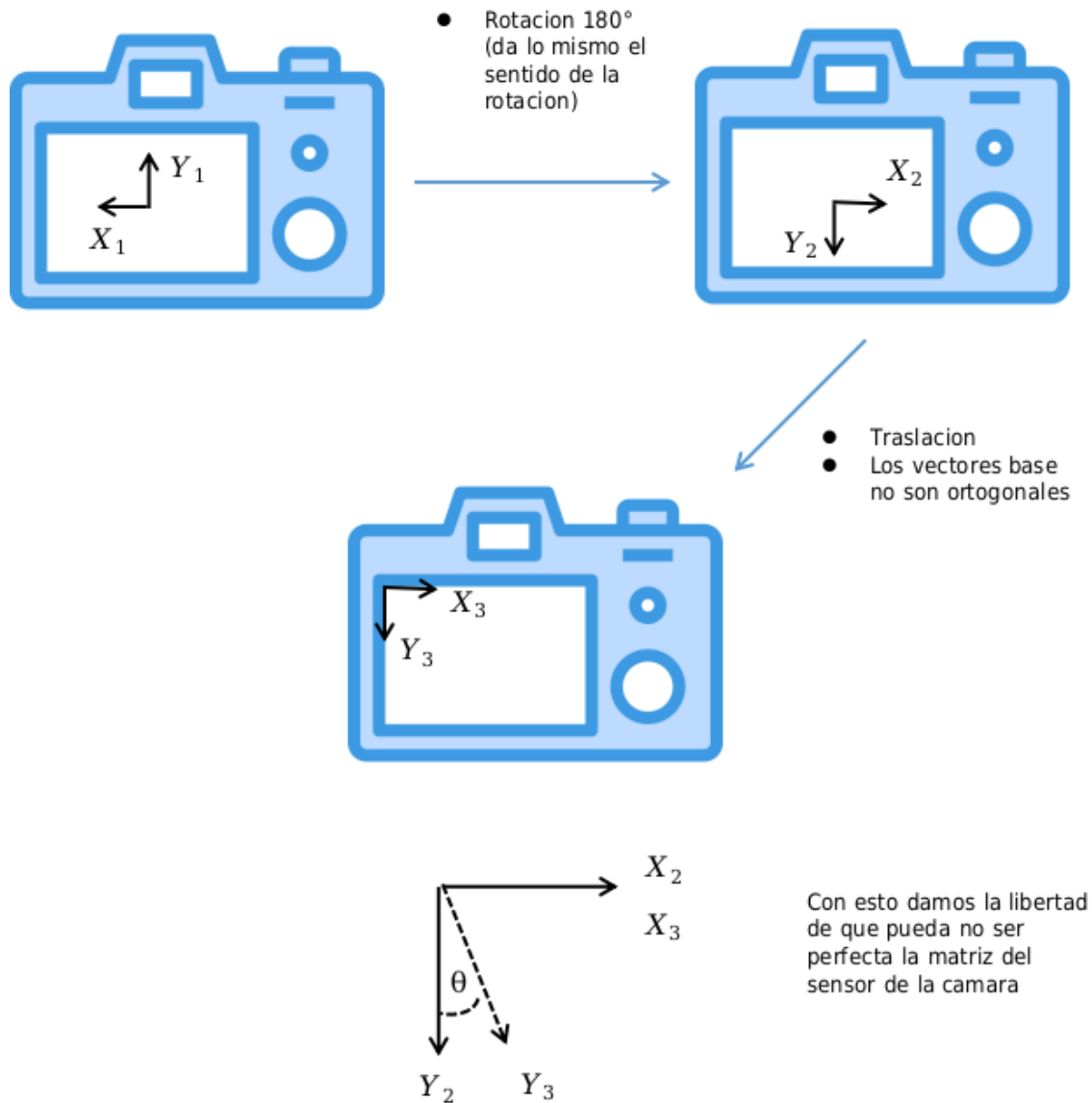


Figura 8. Transformaciones entre sistemas de coordenadas sobre el plano de imagen de la cámara.

En la Figura 8, el sistema de coordenadas 3 sobre el plano de imagen de la cámara está definido de tal manera que coincide con el sistema de coordenadas de la matriz del sensor de la cámara.

Por tanto, se desea encontrar una ecuación que relacione un punto en el sistema de coordenadas 1 con un punto en el sistema de coordenadas 3. Para lograr esto se debe hacer lo siguiente:

- Relación sistema de coordenadas 1 y 2

Se tiene la siguiente ecuación vectorial:

$v_1 = t_{21} + T_{21}v_2$	(12)
----------------------------	--------

En coordenadas homogéneas se expresa de la siguiente manera:

$\begin{bmatrix} v_1 \\ 1 \end{bmatrix} = \begin{bmatrix} T_{21} & t_{21} \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} v_2 \\ 1 \end{bmatrix}$	(13)
--	--------

Donde:

$t_{21} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$	(14)
---	--------

$T_{21} = [-i_1 \quad -j_1] = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$	(15)
---	--------

$v_2 = \begin{bmatrix} v_{2x} \\ v_{2y} \end{bmatrix}$	(16)
--	--------

Por tanto:

$\begin{bmatrix} v_1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_2 \\ 1 \end{bmatrix}$	(17)
---	--------

- Relación sistema de coordenadas 2 y 3

Se tiene la siguiente ecuación vectorial:

$v_2 = t_{32} + T_{32}v_3$	(18)
----------------------------	--------

En coordenadas homogéneas se expresa de la siguiente manera:

$\begin{bmatrix} v_2 \\ 1 \end{bmatrix} = \begin{bmatrix} T_{32} & t_{32} \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} v_3 \\ 1 \end{bmatrix}$	(19)
--	--------

Donde:

$t_{32} = \begin{bmatrix} t_{32x} \\ t_{32y} \end{bmatrix}$	(20)
---	--------

$T_{32} = [i_3 \quad j_3]$	(21)
----------------------------	--------

$v_3 = \begin{bmatrix} v_{3x} \\ v_{3y} \end{bmatrix}$	(22)
--	--------

El análisis para encontrar T_{32} se basa en la Figura 8:

$i_3 = i_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$	(23)
--	--------

$\sin \theta = \frac{j_{3x}}{ j_3 } = \frac{j_{3x}}{1} = j_{3x}$	(24)
--	--------

$\cos \theta = \frac{j_{3y}}{ j_3 } = \frac{j_{3y}}{1} = j_{3y}$	(25)
--	--------

$j_3 = \begin{bmatrix} j_{3x} \\ j_{3y} \end{bmatrix} = \begin{bmatrix} \sin \theta \\ \cos \theta \end{bmatrix}$	(26)
---	--------

Por tanto:

$\begin{bmatrix} v_2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \sin \theta & t_{32x} \\ 0 & \cos \theta & t_{32y} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_3 \\ 1 \end{bmatrix}$	(27)
---	--------

- Relación sistema de coordenadas 1 y 3

Combinando la equ. 17 y 27 se tiene la siguiente ecuación:

$\begin{aligned} \begin{bmatrix} v_1 \\ 1 \end{bmatrix} &= \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_2 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & \sin \theta & t_{32x} \\ 0 & \cos \theta & t_{32y} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_3 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -1 & -\sin \theta & -t_{32x} \\ 0 & -\cos \theta & -t_{32y} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_3 \\ 1 \end{bmatrix} \end{aligned}$	(28)
---	--------

La equ. 28 relaciona un punto del sistema de coordenadas 1 en coordenadas homogéneas con un punto del sistema de coordenadas 3 en coordenadas homogéneas.

Reemplazando la equ. 28 en la equ. 11 se tiene lo siguiente:

$v_{cz} \begin{bmatrix} v_1 \\ 1 \end{bmatrix} = v_{cz} \begin{bmatrix} -1 & -\sin \theta & -t_{32x} \\ 0 & -\cos \theta & -t_{32y} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_3 \\ 1 \end{bmatrix}$ $= \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} v_c$	(29)
--	--------

Manipulando un poco la equ. 29 se tiene lo siguiente:

$v_{cz} \begin{bmatrix} v_3 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & -\sin \theta & -t_{32x} \\ 0 & -\cos \theta & -t_{32y} \\ 0 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} v_c$ $= \begin{bmatrix} -1 & \tan \theta & t_{32y} \tan \theta - t_{32x} \\ 0 & -\sec \theta & -t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} v_c$ $= \begin{bmatrix} -f & f \tan \theta & t_{32y} \tan \theta - t_{32x} \\ 0 & -f \sec \theta & -t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} v_c$	(30)
---	--------

Ahora lo que se busca es expresar el punto sobre el plano de la imagen en unidades de pixeles, no en unidades de milímetros. Para ello se necesita crear las siguientes ecuaciones, donde el punto v_{3p} es un punto expresado en pixeles en el sistema de coordenadas 3:

$v_{3p} = \begin{bmatrix} v_{3px} \\ v_{3py} \end{bmatrix} = \begin{bmatrix} v_{3x} \alpha_x \\ v_{3y} \alpha_y \end{bmatrix} = \begin{bmatrix} \alpha_x & 0 \\ 0 & \alpha_y \end{bmatrix} \begin{bmatrix} v_{3x} \\ v_{3y} \end{bmatrix}$	(31)
$\begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_x & 0 & 0 \\ 0 & \alpha_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_3 \\ 1 \end{bmatrix}$	(32)

Donde:

- $\alpha_x \left(\frac{\text{pixeles}}{\text{milímetros}} \right)$ Factor de conversión de milímetros a pixeles en el eje X

- $\alpha_y \left(\frac{\text{pixeles}}{\text{milímetros}} \right)$ Factor de conversión de milímetros a pixeles en el eje Y

Usando la equ. 30 y 32, y manipulando un poco las ecuaciones, se tiene lo siguiente:

$v_{cz} \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_x & 0 & 0 \\ 0 & \alpha_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -f & f \tan \theta & t_{32y} \tan \theta - t_{32x} \\ 0 & -f \sec \theta & -t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} v_c$ $= \begin{bmatrix} -f\alpha_x & f\alpha_x \tan \theta & \alpha_x(t_{32y} \tan \theta - t_{32x}) \\ 0 & -f\alpha_y \sec \theta & -\alpha_y t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} v_c$	(33)
---	-------

Se define lo siguiente:

$f_x = f\alpha_x$	(34)
$f_y = f\alpha_y$	(35)

Por tanto, la equ. 33 queda de la siguiente manera:

$v_{cz} \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} = \begin{bmatrix} -f_x & f_x \tan \theta & \alpha_x(t_{32y} \tan \theta - t_{32x}) \\ 0 & -f_y \sec \theta & -\alpha_y t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} v_c$	(36)
--	-------

Hasta el momento se tiene una expresión (equ. 36) que relaciona un punto 3D en el sistema de coordenadas de la cámara con un punto 2D en el sistema de coordenadas 3 que se encuentra sobre el plano de imagen. Sin embargo, se quiere expresar el punto 3D con respecto a un sistema de coordenadas conocido, el cual se llamará sistema de coordenadas del mundo. Para ello se tiene la siguiente relación, la cual se muestra en la Figura 9:

$v_c = t_{wc} + T_{wc} v_w$	(37)
$v_c = [T_{wc} \quad t_{wc}] \begin{bmatrix} v_w \\ 1 \end{bmatrix}$	(38)

Donde:

$T_{wc} = [i_w \quad j_w \quad k_w]$	(39)
$t_{wc} = \begin{bmatrix} t_{wcx} \\ t_{wcy} \\ t_{wcz} \end{bmatrix}$	(40)

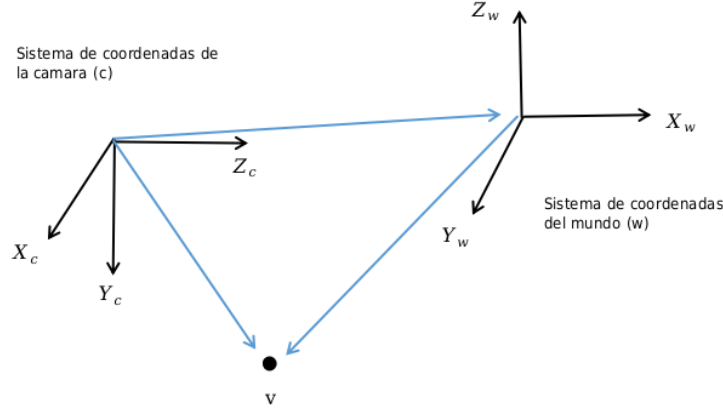


Figura 9. Relación entre el sistema de coordenadas de la cámara y el sistema de coordenadas del mundo.

Combinando la equ. 36 y 38 se tiene lo siguiente:

$v_{cz} \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} = \begin{bmatrix} -f_x & f_x \tan \theta & \alpha_x(t_{32y} \tan \theta - t_{32x}) \\ 0 & -f_y \sec \theta & -\alpha_y t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} [T_{wc} \quad t_{wc}] \begin{bmatrix} v_w \\ 1 \end{bmatrix}$	(41)
---	--------

La equ. 41 corresponde al modelo de cámara Pinhole ideal, el cual fue resultado de un análisis extenso sobre la proyección de un punto 3D sobre el plano de imagen de la cámara.

De la equ. 41 se define lo siguiente:

$m = \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix}$	(42)
$K = \begin{bmatrix} -f_x & f_x \tan \theta & \alpha_x(t_{32y} \tan \theta - t_{32x}) \\ 0 & -f_y \sec \theta & -\alpha_y t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix}$	(43)
$Q = [T_{wc} \quad t_{wc}]$	(44)

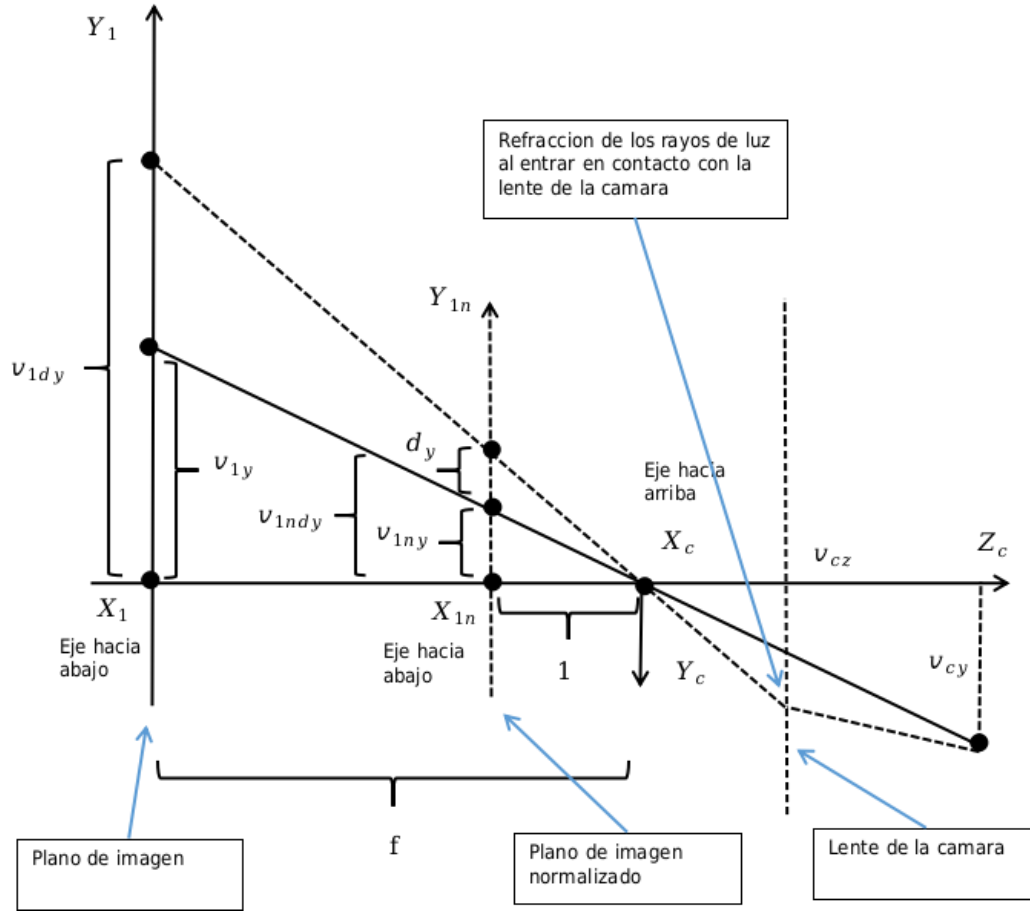


Figura 11. Dibujo en base a la Figura 3 para la obtención de la componente en Y de la proyección del punto 3D sobre el plano de imagen tomando en cuenta la distorsión de la lente de la cámara.

Las ecuaciones resultantes son las siguientes:

$$v_{1nd} = \begin{bmatrix} v_{1ndx} \\ v_{1ndy} \end{bmatrix} = \begin{bmatrix} v_{1nx} + d_x \\ v_{1ny} + d_y \end{bmatrix} = v_{1n} + d \quad (47)$$

Donde:

$$d(v_{1n}) = \begin{bmatrix} d_x(v_{1n}) \\ d_y(v_{1n}) \end{bmatrix} \quad (48)$$

$$= \begin{bmatrix} v_{1nx} (k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 v_{1nx} v_{1ny} + p_2 (r^2 + 2(v_{1nx})^2) \\ v_{1ny} (k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2(v_{1ny})^2) + 2p_2 v_{1nx} v_{1ny} \end{bmatrix}$$

$$r = |v_{1n}| = \sqrt{(v_{1nx})^2 + (v_{1ny})^2} \quad (49)$$

Parámetros de distorsión:

- Radial (k_1, k_2, k_3)
- Tangencial (p_1, p_2)

El punto v_{1nd} es un punto distorsionado en el plano de imagen normalizado y $d(v_{1n})$ es un vector de distorsion en el plano de imagen normalizado que depende del punto v_{1n} .

Es importante mencionar que el análisis con distorsión se lleva a cabo de tal manera que se asume que la distorsión de la lente no depende de la distancia focal de la cámara.

Ahora, tomando en cuenta la equ. 5, es posible definir la siguiente expresión:

$v_{1d} = fv_{1nd}$	(50)
$\begin{bmatrix} v_{1d} \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix}$	(51)

Punto plano de imagen = (f)(Punto plano de imagen normalizado)

El punto v_{1d} es un punto distorsionado en el sistema de coordenadas 1.

A continuación se encontrará la expresión correspondiente para el modelo de cámara Pinhole no ideal. La equ. 28 quedaría de la siguiente manera tomando en cuenta la distorsión:

$\begin{bmatrix} v_{1d} \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & -\sin \theta & -t_{32x} \\ 0 & -\cos \theta & -t_{32y} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{3d} \\ 1 \end{bmatrix}$	(52)
--	-------

Combinando la equ. 51 y 52, y manipulando un poco la expresión, se tiene lo siguiente:

$ \begin{aligned} \begin{bmatrix} v_{3d} \\ 1 \end{bmatrix} &= \begin{bmatrix} -1 & -\sin \theta & -t_{32x} \\ 0 & -\cos \theta & -t_{32y} \\ 0 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -1 & \tan \theta & t_{32y} \tan \theta - t_{32x} \\ 0 & -\sec \theta & -t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -f & f \tan \theta & t_{32y} \tan \theta - t_{32x} \\ 0 & -f \sec \theta & -t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} \end{aligned} $	(53)
---	--------

Ahora lo que se busca es expresar el punto sobre el plano de imagen en unidades de pixeles, no en unidades de milímetros. Para ello se necesita crear las siguientes ecuaciones, donde el punto v_{3pd} es un punto distorsionado expresado en pixeles en el sistema de coordenadas 3:

$v_{3pd} = \begin{bmatrix} v_{3pdx} \\ v_{3pdy} \end{bmatrix} = \begin{bmatrix} v_{3dx} \alpha_x \\ v_{3dy} \alpha_y \end{bmatrix} = \begin{bmatrix} \alpha_x & 0 \\ 0 & \alpha_y \end{bmatrix} \begin{bmatrix} v_{3dx} \\ v_{3dy} \end{bmatrix}$	(54)
$\begin{bmatrix} v_{3pd} \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_x & 0 & 0 \\ 0 & \alpha_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{3d} \\ 1 \end{bmatrix}$	(55)

Combinando la equ. 53 y 55 se tiene lo siguiente:

$ \begin{aligned} \begin{bmatrix} v_{3pd} \\ 1 \end{bmatrix} &= \begin{bmatrix} \alpha_x & 0 & 0 \\ 0 & \alpha_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -f & f \tan \theta & t_{32y} \tan \theta - t_{32x} \\ 0 & -f \sec \theta & -t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -f \alpha_x & f \alpha_x \tan \theta & \alpha_x (t_{32y} \tan \theta - t_{32x}) \\ 0 & -f \alpha_y \sec \theta & -\alpha_y t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -f_x & f_x \tan \theta & \alpha_x (t_{32y} \tan \theta - t_{32x}) \\ 0 & -f_y \sec \theta & -\alpha_y t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} \\ &= K \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} \end{aligned} $	(56)
--	--------

La equ. 56 corresponde al modelo de cámara Pinhole no ideal.

De la equ. 56 se define lo siguiente:

$m_d = \begin{bmatrix} v_{3pd} \\ 1 \end{bmatrix}$	(57)
$M_{nd} = \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix}$	(58)

Por tanto, la equ. 56 queda resumida en la siguiente expresión:

$m_d = KM_{nd}$	(59)
-----------------	-------

Recordar que el punto v_{1nd} esta en funcion de los parametros de distorsion y del punto 3D con respecto al sistema de coordenadas de la camara (c), el cual a su vez puede ser representado en el sistema de coordenadas del mundo (w) mediante la equ. 38.

Hasta aquí se ha deducido el modelo de cámara Pinhole ideal (equ. 41) y el no ideal (equ. 56).

Con respecto a la terminología que se da a algunas variables:

- K: Matriz intrínseca (equ. 43)

Esta matriz se calcula una sola vez por cámara.

- Q: Matriz extrínseca (equ. 44)

Esta matriz se calcula cada vez que el sistema de coordenadas del mundo (w) haya cambiado con respecto al sistema de coordenadas de la cámara (c).

Por lo general se suele definir la siguiente variable, solo por simplicidad, la cual representa los parámetros de distorsión de la cámara:

$q = \begin{bmatrix} k_1 \\ k_2 \\ k_3 \\ p_1 \\ p_2 \end{bmatrix}$	(60)
---	-------

Mas adelante, en el proceso de calibración de cámara, se detallará como se puede calcular la matriz intrínseca (K) y los parámetros de distorsión (q).

La Figura 12 muestra la correspondencia de los puntos sobre una imagen y el punto v_{3pd} .

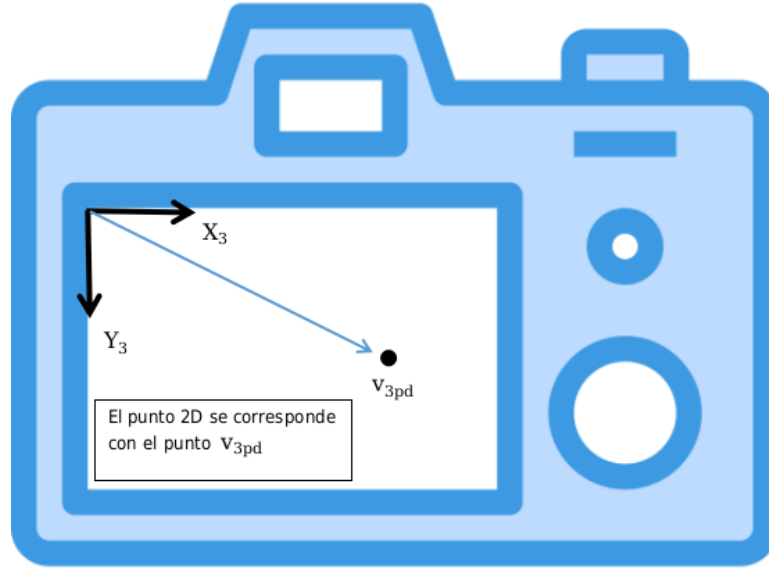


Figura 12. Punto 2D distorsionado sobre el plano de imagen de la cámara.

Dados los modelos ideal y no ideal, es posible encontrar el punto 2D no distorsionado a partir del punto 2D distorsionado. Para ello es muy importante contar con la matriz intrínseca (K) y los parámetros de distorsión (q) de la cámara. Los pasos se muestran a continuación:

- 1) Se manipula la equ. 47 hasta obtener una función que dependa de v_{1n} que este igualada a 0:

$$f(v_{1n}) = v_{1nd} - v_{1n} - d(v_{1n}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (61)$$

- 2) El valor para v_{1nd} proviene de la equ. 59 despejando M_{nd} :

$$M_{nd} = \begin{bmatrix} v_{1nd} \\ 1 \end{bmatrix} = K^{-1}m_d = K^{-1} \begin{bmatrix} v_{3pd} \\ 1 \end{bmatrix} \quad (62)$$

- 3) Se establece un problema de búsqueda de raíces para la función $f(v_{1n})$ con la intención de encontrar el valor de v_{1n} que cumpla la equ. 61. Como valor inicial para

v_{1n} sera el valor de v_{1nd} , ya que se supondra que $v_{1n} \approx v_{1nd}$, esto dado la equ. 47 y tomando en cuenta que el vector de distorsión es pequeño.

Para resolver el problema de búsqueda de raíces se utiliza el método *root* del módulo *scipy.optimize* de Python.

- 4) Una vez obtenido v_{1n} se puede utilizar el modelo ideal para calcular el punto 2D sin distorsion. Para ello se toma la equ. 7 y se pasa a coordenadas homogéneas (equ. 63). Después se multiplica ambos lados de la ecuación por v_{cz} (equ. 64). Por ultimo, se sustituye la equ. 64 en la equ. 36 (lo que resulta en la equ. 65).

$\begin{bmatrix} v_{1n} \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{v_{cx}}{v_{cz}} \\ \frac{v_{cy}}{v_{cz}} \\ 1 \end{bmatrix}$	(63)
$v_{cz} \begin{bmatrix} v_{1n} \\ 1 \end{bmatrix} = \begin{bmatrix} v_{cx} \\ v_{cy} \\ v_{cz} \end{bmatrix} = v_c$	(64)
$v_{cz} \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} = K v_c = K v_{cz} \begin{bmatrix} v_{1n} \\ 1 \end{bmatrix}$ $\begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} = K \begin{bmatrix} v_{1n} \\ 1 \end{bmatrix}$	(65)

En la práctica se utiliza el modelo no ideal para eliminar la distorsión de puntos 2D y después se usa el modelo ideal con puntos 2D sin distorsión.

4.2. Calibración de cámara

La calibración de la cámara es un paso muy importante para poder llevar a cabo tareas como lo es la estimación de puntos 3D característicos del cuerpo. Esta calibración consiste en calcular los parámetros intrínsecos y de distorsión de la cámara, y tan solo requiere que se haga una sola vez por cámara. Gran parte de los pasos de calibración están basados en [9,10].

De las técnicas clásicas de calibración se tienen las siguientes:

- Direct Linear Transformation (DLT): Requiere de un equipo de calibración 3D (se requieren coordenadas 3D extremadamente precisas)
- Multiplane Calibration (método de Zhang): Requiere capturar múltiples vistas de una única superficie plana

El método de calibración que se utilizó está basado en el método de Zhang [11].

Los parámetros de la cámara que se desean estimar son los siguientes:

- Parámetros intrínsecos (K)
- Parámetros de distorsión (q)

Para encontrar los parámetros óptimos de la cámara se define un problema de optimización, en la cual se busca minimizar el error entre un punto 2D distorsionado dado por el modelo no ideal y un punto 2D distorsionado dado por un algoritmo de detección de puntos 2D sobre una imagen. Se habla de puntos distorsionados porque provienen de una imagen distorsionada. Para el problema es necesario una buena inicialización de los parámetros dada la complejidad del problema. El análisis siguiente se centrará en encontrar una buena inicialización de los parámetros. Del problema de optimización se hablará más adelante.

Lo primero que se hará es encontrar una aproximación de los parámetros intrínsecos (K) de la cámara mediante el modelo de cámara Pinhole ideal. Para ello se asumirá que la cámara es perfecta, por tanto, cada punto 2D en la imagen tomada por la cámara corresponderá al punto v_{3p} en el modelo ideal (y no al punto v_{3pd} como en realidad sería). La imagen se supondrá no esta distorsionada.

Se empezará tomando la equ. 41 del modelo ideal y se establecerá $v_{wz} = 0$, ya que se estará trabajando con puntos 3D en el sistema de coordenadas del mundo que viven en el plano X_wY_w . Esto permite simplificar la expresión. La expresión resultante queda de la siguiente manera:

$ \begin{aligned} v_{cz} \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} &= \begin{bmatrix} -f_x & f_x \tan \theta & \alpha_x(t_{32y} \tan \theta - t_{32x}) \\ 0 & -f_y \sec \theta & -\alpha_y t_{32y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix} [T_{wc} \quad t_{wc}] \begin{bmatrix} v_w \\ 1 \end{bmatrix} \\ &= K[i_w \quad j_w \quad k_w \quad t_{wc}] \begin{bmatrix} v_{wx} \\ v_{wy} \\ v_{wz} \\ 1 \end{bmatrix} = K[i_w \quad j_w \quad k_w \quad t_{wc}] \begin{bmatrix} v_{wx} \\ v_{wy} \\ 0 \\ 1 \end{bmatrix} \\ &= K[i_w \quad j_w \quad t_{wc}] \begin{bmatrix} v_{wx} \\ v_{wy} \\ 1 \end{bmatrix} \end{aligned} $	(66)
---	-------

De lo anterior se define lo siguiente:

$Q^* = [i_w \quad j_w \quad t_{wc}]$	(67)
$M^* = \begin{bmatrix} v_{wx} \\ v_{wy} \\ 1 \end{bmatrix}$	(68)

Quedando la equ. 66 de la siguiente manera:

$v_{cz}m = KQ^*M^*$	(69)
---------------------	-------

Ahora se define lo siguiente:

$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} = KQ^*$	(70)
---	-------

Quedando la equ. 69 de la siguiente manera:

$v_{cz}m = HM^*$	(71)
------------------	-------

La equ. 71 se multiplica a ambos lados de la ecuación por $\frac{1}{h_{33}}$, lo cual da como resultado lo siguiente:

$\frac{1}{h_{33}}v_{cz}m = \frac{1}{h_{33}}HM^*$	(72)
--	-------

Se define lo siguiente:

$\alpha = \frac{1}{h_{33}} v_{cz}$	(73)
$G = \frac{1}{h_{33}} H = \begin{bmatrix} g_{11} & g_{12} & g_{13} \\ g_{21} & g_{22} & g_{23} \\ g_{31} & g_{32} & 1 \end{bmatrix}$	(74)

La ecuación final resultante es la siguiente:

$\alpha m = GM^*$	
$\alpha \begin{bmatrix} v_{3px} \\ v_{3py} \\ 1 \end{bmatrix} = \begin{bmatrix} g_{11} & g_{12} & g_{13} \\ g_{21} & g_{22} & g_{23} \\ g_{31} & g_{32} & 1 \end{bmatrix} \begin{bmatrix} v_{wx} \\ v_{wy} \\ 1 \end{bmatrix}$	(75)

Ahora, partiendo de la equ. 75, es posible calcular la matriz G mediante pares de puntos correspondientes 2D (en pixeles) y 3D (en mm), así como se muestra en la Figura 13. Por cada par de puntos correspondientes se obtienen 2 ecuaciones, por tanto, para calcular a la matriz G (la cual tiene 8 incógnitas) se necesitan por lo menos 4 pares de puntos correspondientes. El cálculo de G servirá más adelante. Para calcular G se utiliza la equ. 75, de la cual se obtienen las siguientes ecuaciones:

$\alpha v_{3px} = g_{11} v_{wx} + g_{12} v_{wy} + g_{13}$	(76)
$\alpha v_{3py} = g_{21} v_{wx} + g_{22} v_{wy} + g_{23}$	(77)
$\alpha = g_{31} v_{wx} + g_{32} v_{wy} + 1$	(78)

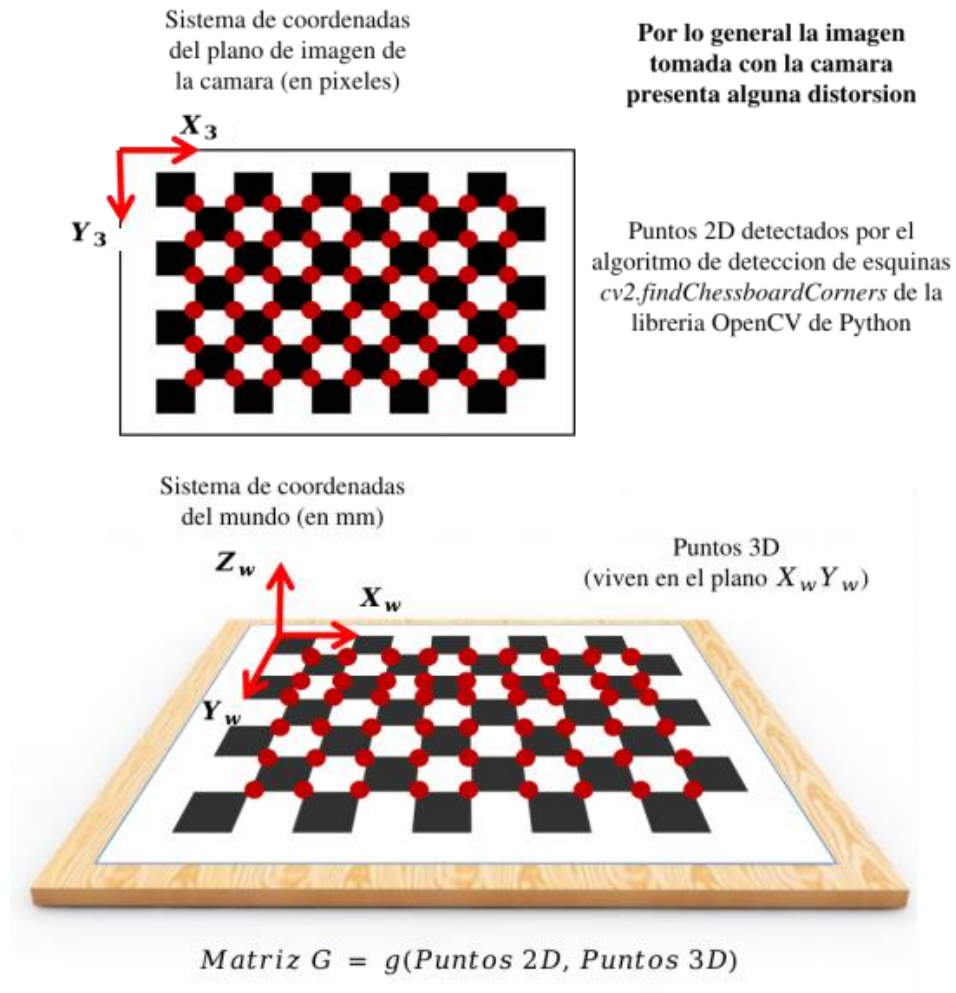


Figura 13. Obtención de pares de puntos correspondientes 2D y 3D a partir de un tablero de ajedrez.

Usando la equ. 78 en la equ. 76 y 77, y manipulando un poco las expresiones, se obtiene lo siguiente:

$g_{11}v_{wx} + g_{12}v_{wy} + g_{13} - g_{31}v_{wx}v_{3px} - g_{32}v_{wy}v_{3px} = v_{3px}$	(79)
$g_{21}v_{wx} + g_{22}v_{wy} + g_{23} - g_{31}v_{wx}v_{3py} - g_{32}v_{wy}v_{3py} = v_{3py}$	(80)

Las ecuaciones anteriores en forma matricial quedarían de la siguiente manera:

$\begin{bmatrix} v_{wx} & v_{wy} & 1 & 0 & 0 & 0 & -v_{wx}v_{3px} & -v_{wy}v_{3px} \\ 0 & 0 & 0 & v_{wx} & v_{wy} & 1 & -v_{wx}v_{3py} & -v_{wy}v_{3py} \end{bmatrix} \begin{bmatrix} g_{11} \\ g_{12} \\ g_{13} \\ g_{21} \\ g_{22} \\ g_{23} \\ g_{31} \\ g_{32} \end{bmatrix} = \begin{bmatrix} v_{3px} \\ v_{3py} \end{bmatrix}$	(81)
--	-------

Como se puede observar, mínimo se necesitan 4 pares de puntos correspondientes 2D (en pixeles) y 3D (en mm) para que el sistema de ecuaciones pueda tener solución.

Es posible automatizar la obtención de pares de puntos correspondientes utilizando un tablero de ajedrez, así como se muestra en la Figura 13. El procedimiento es el siguiente:

- Los puntos 2D se obtienen utilizando algún método de detección de puntos en el tablero de ajedrez. El método que se utilizó es *cv2.findChessboardCorners* de la librería OpenCV. Para utilizar el método se necesita conocer las dimensiones del tablero de ajedrez.
- Los puntos 3D deben ser correspondientes con los puntos 2D anteriormente obtenidos. Se debe medir manualmente la longitud de un cuadrado (en mm) del tablero de ajedrez para automatizar el proceso de obtención de puntos 3D. El sistema de coordenadas del mundo (w) estará colocado sobre el tablero de ajedrez como se muestra en la Figura 13 y dichos puntos deben corresponder a dicho sistema de coordenadas.

Por tanto, las ecuaciones generadas por cada par de puntos correspondientes se agregan para obtener un sistema de ecuaciones de la siguiente forma:

$Ax = b$	(82)
----------	-------

Siendo N el número de pares de puntos correspondientes ($N \geq 4$), se tiene que:

- $A \in \mathbb{R}^{2N \times 8}$
- $x \in \mathbb{R}^8$
- $b \in \mathbb{R}^{2N \times 1}$

La solución para la equ. 82 es la siguiente (para más información consultar el anexo 1):

$$x = (A^T A)^{-1} A^T b = \begin{bmatrix} g_{11} \\ g_{12} \\ g_{13} \\ g_{21} \\ g_{22} \\ g_{23} \\ g_{31} \\ g_{32} \end{bmatrix} \quad (83)$$

La equ. 83 permite construir la matriz G. Teniendo la matriz G se procede a construir las siguientes ecuaciones en base a la equ. 74:

$[g_1 \ g_2 \ g_3] = \frac{1}{h_{33}} [h_1 \ h_2 \ h_3] = \frac{1}{h_{33}} K [i_w \ j_w \ t_{wc}]$ $= \left[\frac{1}{h_{33}} K i_w \quad \frac{1}{h_{33}} K j_w \quad \frac{1}{h_{33}} K t_{wc} \right]$	(84)
$g_1 = \frac{1}{h_{33}} K i_w$ $i_w = h_{33} K^{-1} g_1$	(85)
$g_2 = \frac{1}{h_{33}} K j_w$ $j_w = h_{33} K^{-1} g_2$	(86)
$g_3 = \frac{1}{h_{33}} K t_{wc}$ $t_{wc} = h_{33} K^{-1} g_3$	(87)
$k_w = -(i_w \times j_w)$	(88)

Como adición se agrega la equ. 88, la cual permite calcular k_w a partir de i_w y j_w . El producto vectorial sigue la regla de la mano derecha.

Las ecuaciones anteriores brindan una manera de calcular la matriz extrínseca (Q) a partir de G y de la matriz intrínseca (K). Ya que se obtenga la matriz intrínseca (K) se volverá a estas

ecuaciones para calcular la matriz extrínseca (Q) correspondiente. Recordar que estas matrices son indispensables en el modelo de cámara Pinhole ideal.

Antes de continuar, es importante mencionar las restricciones que se deben tomar en cuenta para la matriz extrínseca (Q), recordando que los vectores base del sistema de coordenadas del mundo (w) forma una base ortonormal de vectores:

$ i_w = 1$ $i_w^T i_w = 1$	(89)
$ j_w = 1$ $j_w^T j_w = 1$	(90)
$ k_w = 1$ $k_w^T k_w = 1$	(91)
$i_w^T j_w = 0$	(92)
$i_w^T k_w = 0$	(93)
$j_w^T k_w = 0$	(94)

Tomando la equ. 85 con la 89 y la equ. 86 con la 90, es posible determinar h_{33} obteniendo dos soluciones de la siguiente manera:

$ i_w = h_{33}K^{-1}g_1 = h_{33} K^{-1}g_1 = 1$ $h_{33} = \frac{1}{ K^{-1}g_1 }$	(95)
$ j_w = h_{33}K^{-1}g_2 = h_{33} K^{-1}g_2 = 1$ $h_{33} = \frac{1}{ K^{-1}g_2 }$	(96)

Combinando la equ. 89 y 90, y sustituyendo la equ. 85 y 86, se tiene lo siguiente:

$\begin{aligned} i_w^T i_w &= j_w^T j_w \\ (h_{33}K^{-1}g_1)^T(h_{33}K^{-1}g_1) &= (h_{33}K^{-1}g_2)^T(h_{33}K^{-1}g_2) \\ g_1^T(K^{-1})^TK^{-1}g_1 - g_2^T(K^{-1})^TK^{-1}g_2 &= 0 \end{aligned}$	(97)
--	-------

Usando la equ. 92, y sustituyendo la equ. 85 y 86, se tiene lo siguiente:

$\begin{aligned} i_w^T j_w &= 0 \\ (h_{33}K^{-1}g_1)^T(h_{33}K^{-1}g_2) &= 0 \\ g_1^T(K^{-1})^TK^{-1}g_2 &= 0 \end{aligned}$	(98)
--	-------

Tomando en cuenta la equ. 97 y 98, se define lo siguiente:

$B = (K^{-1})^TK^{-1} = \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}$	(99)
--	-------

La matriz B de la equ. 99 es una matriz simétrica, lo cual significa que $B = B^T$. Esto es debido a lo siguiente:

- La matriz K es una matriz triangular superior
- K^{-1} es una matriz triangular superior
- $(K^{-1})^T$ es una matriz triangular inferior
- $(K^{-1})^TK^{-1}$ es una matriz simétrica

Notar que si se encuentra la matriz B es posible encontrar la matriz K con algunos pasos adicionales.

La equ. 97 y 98 quedan de la siguiente manera:

$\begin{aligned} g_1^TBg_1 - g_2^TBg_2 &= 0 \\ a(g_{11}^2 - g_{12}^2) + b(2g_{11}g_{21} - 2g_{12}g_{22}) + c(2g_{11}g_{31} - 2g_{12}g_{32}) + d(g_{21}^2 - g_{22}^2) \\ + e(2g_{21}g_{31} - 2g_{22}g_{32}) + f(g_{31}^2 - g_{32}^2) &= 0 \end{aligned}$	(100)
$\begin{aligned} g_1^TBg_2 &= 0 \\ a(g_{11}g_{12}) + b(g_{12}g_{21} + g_{11}g_{22}) + c(g_{12}g_{31} + g_{11}g_{32}) + d(g_{21}g_{22}) + e(g_{22}g_{31} + g_{21}g_{32}) \\ + f(g_{31}g_{32}) &= 0 \end{aligned}$	(101)

En forma matricial la equ. 100 y 101 se pueden expresar de la siguiente manera:

$\begin{bmatrix} g_{11}^2 - g_{12}^2 & 2g_{11}g_{21} - 2g_{12}g_{22} & 2g_{11}g_{31} - 2g_{12}g_{32} & g_{21}^2 - g_{22}^2 & 2g_{21}g_{31} - 2g_{22}g_{32} & g_{31}^2 - g_{32}^2 \\ g_{11}g_{12} & g_{12}g_{21} + g_{11}g_{22} & g_{12}g_{31} + g_{11}g_{32} & g_{21}g_{22} & g_{22}g_{31} + g_{21}g_{32} & g_{31}g_{32} \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$ $= \begin{bmatrix} 0 \\ 0 \end{bmatrix}$	(102)
---	---------

La matriz B se puede calcular mediante matrices G. Por cada matriz G se obtienen 2 ecuaciones para 6 incógnitas. Se necesitan al menos 3 matrices G para que el sistema de ecuaciones pueda tener solución. La Figura 13 muestra cómo obtener una matriz G. Cada matriz G está asociada a una imagen tomada por la cámara y a pares de puntos correspondientes 2D y 3D. Por tanto, se deben tomar varias imágenes del tablero de ajedrez en diferentes orientaciones para obtener diferentes matrices G. La variedad en las orientaciones y posiciones del tablero de ajedrez es importante para obtener matrices G diferentes.

Las ecuaciones generadas por cada matriz G se agregan para obtener un sistema de ecuaciones de la siguiente forma:

$Ax = 0$	(103)
----------	---------

Siendo S el número de matrices G ($S \geq 3$), se tiene que:

- $A \in \mathbb{R}^{2S \times 6}$
- $x \in \mathbb{R}^6$
- $0 \in \mathbb{R}^{2S \times 1}$

La solución para la equ. 103 es el vector propio con el valor propio asociado más pequeño del siguiente problema (para más información consultar el anexo 2):

$A^T A x = \lambda x \rightarrow Cx = \lambda x$	(104)
--	---------

$x = \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$	
--	--

La equ. 104 permite construir la matriz B. Teniendo la matriz B es posible calcular la matriz intrínseca (K) de la cámara haciendo uso de la descomposición de Cholesky [12]. La descomposición de Cholesky permite descomponer una matriz simétrica definida positiva en la multiplicación de una matriz triangular inferior y la transpuesta de la matriz triangular inferior. No se asegura que la matriz B sea siempre una matriz definida positiva, por lo que si falla la descomposición de Cholesky sobre B, entonces se deberá volver a repetir el proceso de captura de imágenes del tablero de ajedrez y todos los pasos mencionados anteriormente hasta llegar a este punto de nuevo.

La descomposición de Cholesky sobre B queda de la siguiente manera:

$\text{Cholesky}(B) = B = LL^T$	(105)
---------------------------------	--------

De la equ. 105 se define la siguiente expresión:

$L = (K^{-1})^T$	(106)
------------------	--------

Notar que la equ. 106 es verdadera debido a que se cumple que $LL^T = (K^{-1})^TK^{-1} = B$.

Por tanto, se tiene lo siguiente:

$K = (L^T)^{-1} = \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ 0 & k_{22} & k_{23} \\ 0 & 0 & k_{33} \end{bmatrix}$	(107)
--	--------

Es importante mencionar que aunque por naturaleza de la matriz K de que $k_{33} = 1$, el valor de k_{33} para la matriz K calculada no sera necesariamente 1. Esto es asi debido a que se ha hecho el análisis suponiendo que la camara era perfecta, pero se sabe que no es asi. Lo unico que se puede decir es que la matriz K calculada es una aproximacion a la matriz K verdadera.

Ya finalizado el proceso de la obtención de la matriz intrínseca (K) de la cámara, lo que sigue es calcular las matrices extrínsecas (Q's) para cada matriz G, esto es, la matriz extrínseca Q para cada imagen tomada del tablero de ajedrez en diferentes orientaciones, lo cual es posible debido a la equ. 85, 86, 87 y 88, junto con la equ. 95 y 96.

En este punto ya se tiene calculada la matriz intrínseca (K) de la cámara y las matrices extrínsecas (Q's) asociadas a cada imagen tomada del tablero de ajedrez.

Ahora lo que sigue es el proceso de refinamiento de los parámetros de la cámara, el cual consiste en obtener los parámetros reales resolviendo un problema de optimización con restricciones en donde se requiere minimizar una función objetivo, la cual mide el error entre un punto 2D predicho por el modelo de cámara Pinhole no ideal y un punto 2D real dado por el algoritmo de detección de esquinas del tablero de ajedrez, el cual se menciona en la Figura 13. La información de los puntos 2D reales que se usan como datos de referencia son los datos que se usaron en la obtención de las matrices G.

La función a minimizar es la siguiente:

$f(K, q, Q^{*(1)}, \dots, Q^{*(S)}) = \sum_{s=1}^S \sum_{n=1}^N m_d^{(s,n)}(K, q, Q^{*(s)}, M^{*(s,n)}) - m_{dr}^{(s,n)} ^2$	(108)
---	--------

Donde:

- N: Numero de pares de puntos correspondientes que se utilizaron para calcular la matriz G
- S: Numero de matrices G calculadas
- K: Corresponde a la equ. 43
- q: Corresponde a la equ. 60
- $Q^{*(s)}$: Corresponde a la equ. 67
- $M^{*(s,n)}$: Corresponde a la equ. 68

- $m_d^{(s,n)}$: Corresponde a la equ. 59
- $m_{dr}^{(s,n)}$: Corresponde al punto 2D detectado por el algoritmo de deteccion de esquinas del tablero de ajedrez

También se impondrán las restricciones de la equ. 89, 90 y 92, para cada matriz Q^* , esto con el objetivo de asegurar que los vectores i_w y j_w formen un conjunto de vectores ortonormales. Ya que eso deben cumplir.

Por tanto, la función lagrangiana resultante queda de la siguiente manera:

$ \begin{aligned} &L(K, q, Q^{*(1)}, \dots, Q^{*(S)}, \lambda^{(1,1)}, \dots, \lambda^{(1,3)}, \dots, \lambda^{(S,1)}, \dots, \lambda^{(S,3)}) \\ &= f(K, q, Q^{*(1)}, \dots, Q^{*(S)}) \\ &\quad - \sum_{s=1}^S \left(\lambda^{(s,1)} \left(i_w^{T(s)} i_w^{(s)} - 1 \right) + \lambda^{(s,2)} \left(j_w^{T(s)} j_w^{(s)} - 1 \right) + \lambda^{(s,3)} \left(i_w^{T(s)} j_w^{(s)} \right) \right) \end{aligned} $	(109)
---	--------

Valores iniciales:

- K : La matriz K calculada anteriormente

Dado que para la matriz K calculada se tiene que $K_{33} \neq 1$, una mejor inicialización para la matriz K es hacer que $K = \frac{1}{K_{33}} K$, para que la matriz nueva cumpla con que $K_{33} = 1$. Esto ayuda mucho a la convergencia en la búsqueda de los valores optimos de la funcion lagrangiana.

- q : Se inicializa a cero
- $Q^{*(s)}$: El valor de $Q^{*(s)}$ calculado anteriormente
- λ : Se inicializa a cero (para cada λ)

Como optimizador para la función lagrangiana se utiliza el algoritmo de Adam, el cual es muy utilizado para optimizar redes neuronales (para más información consultar el anexo 3). Las derivadas de la función lagrangiana están calculadas a mano y se implementan en la aplicación de Python.

Recordar que el vector k_w es posible calcularlo a partir de la equ. 88. Esto sera necesario despues de encontrar los parametros optimos para cada matriz Q^* , para despues construir la matriz Q .

Al final, los parámetros óptimos son los siguientes:

- K : Matriz intrínseca
- q : Parámetros de distorsión
- Q 's: Matrices extrínsecas
- λ 's: Multiplicadores de lagrange

Es importante guardar los parámetros anteriores para su posterior uso, en especial K y q , ya que son únicos para la cámara y solo se calculan una sola vez. En este punto ya se puede decir que la cámara esta calibrada.

Posteriormente se necesitará calcular la matriz extrínseca (Q) cuando se desee realizar estimaciones de puntos 3D característicos del cuerpo humano. Para ello se deben seguir los siguientes pasos:

- 1) Se deben obtener pares de puntos correspondientes 2D con distorsión y 3D.
- 2) Para cada punto 2D distorsionado, se utiliza la función de la equ. 61 para encontrar las raíces de dicha función y posteriormente se usa la equ. 65 para encontrar el punto 2D sin distorsión.

Es posible definir una función que englobe los pasos anteriores de la siguiente forma:

$v_{3p} = \text{undistorted}(v_{3pd}, K, q)$	(110)
--	---------

- 3) Usando el modelo ideal, se calcula la matriz G correspondiente utilizando los pares de puntos correspondientes 2D sin distorsión y 3D.

- 4) Se calcula Q a partir de G y de la matriz intrínseca (K)

Es posible definir una función que englobe los pasos anteriores de la siguiente forma:

$Q = h(G, K)$	(111)
---------------	--------

Recordar que la matriz extrínseca (Q) se debe calcular cada vez que se desea definir el sistema de coordenadas del mundo (w). Por tanto, este proceso se debe hacer cada vez que se muevan las cámaras o se quiera cambiar el sistema de coordenadas del mundo (w).

En la aplicación de Python, los parámetros de calibración que se necesitan especificar se muestran en la Figura 14. Estos parámetros son necesarios para configurar todo lo necesario para la calibración de la cámara.

Dimensiones tablero (w,h) Ejemplo: 9,6 9,6	Dimensiones cuadrado (mm) 25.0
Numero de imagenes (minimo 3) 5	Tiempo entre captura (s) 3

Figura 14. Parámetros requeridos en la aplicación de Python para la calibración de la cámara.

La Figura 15 muestra a que se refieren las dimensiones del tablero de ajedrez. Las dimensiones del cuadrado es la longitud de un solo cuadrado (en milímetros).

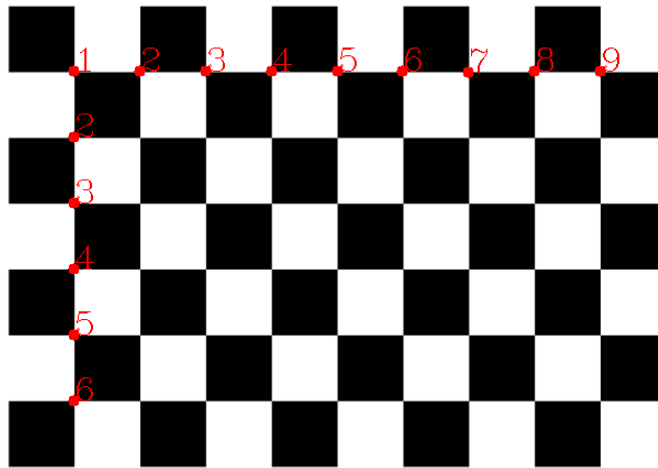


Figura 15. Ejemplo tablero de ajedrez de dimensiones 9,6: 9 esquinas interiores de ancho y 6 esquinas interiores de largo.

Un ejemplo de calibración de cámara se muestra en la Figura 16.

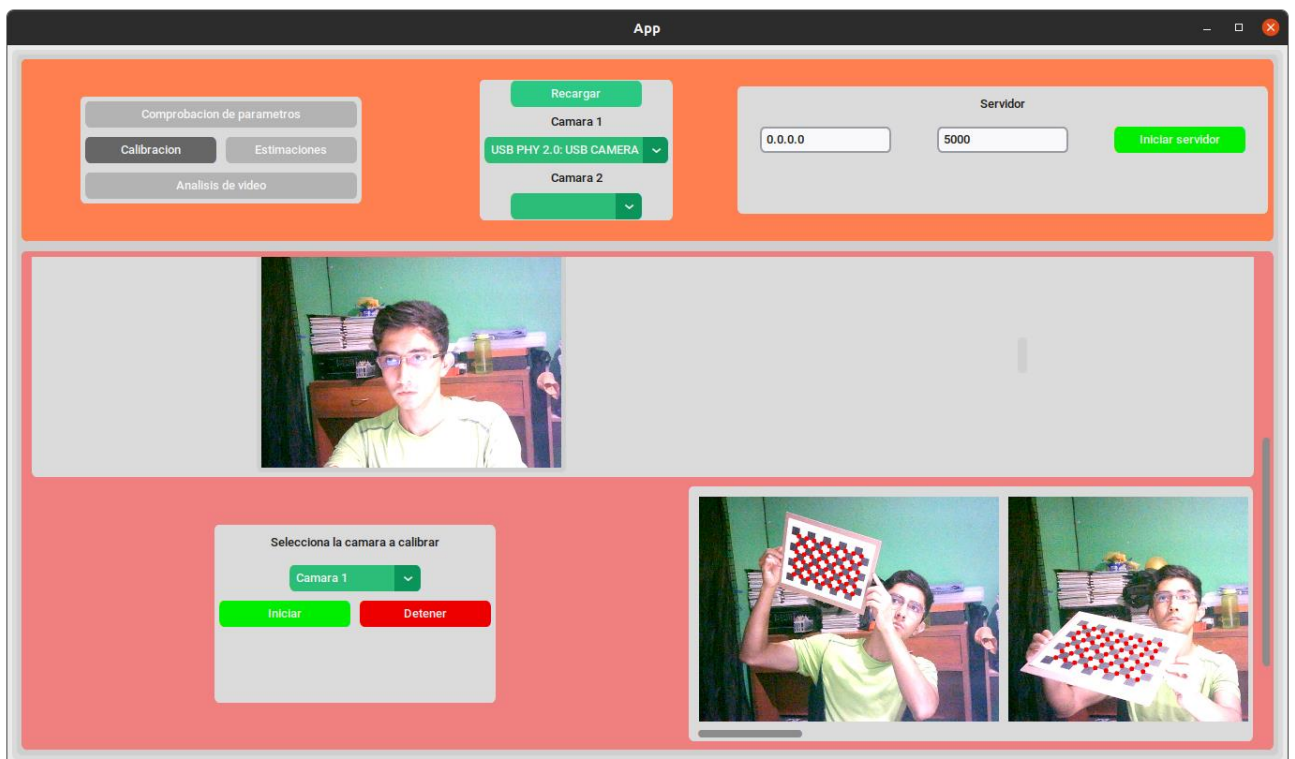


Figura 16. Calibración de cámara en la aplicación de Python.

4.3. Comprobación de los parámetros de la cámara

Después de obtener los parámetros óptimos de la cámara en el proceso de calibración, es muy importante comprobar que los parámetros estén bien calculados, y para ello una buena práctica es hacer un poco de realidad aumentada sobre el tablero de ajedrez con el que se realizó la calibración.

Para realizar realidad aumentada es necesario utilizar el modelo de cámara Pinhole no ideal, el cual se encuentra simplificado en la equ. 59. Para obtener un punto 2D (distorsionado) se necesita lo siguiente:

- Matriz intrínseca (K)
- Parámetros de distorsión (q)
- Matriz extrínseca (Q)
- Punto 3D (en mm)

Como la matriz extrínseca (Q) esta sobre el tablero de ajedrez, y la realidad aumentada es en tiempo real, la matriz Q se estará calculando en cada instante de tiempo. Los puntos 3D vendrán de los vértices de un modelo 3D los cuales se encuentran definidos en el sistema de coordenadas del mundo (w).

Por simplicidad se dibuja un cubo como se muestra en la Figura 17.

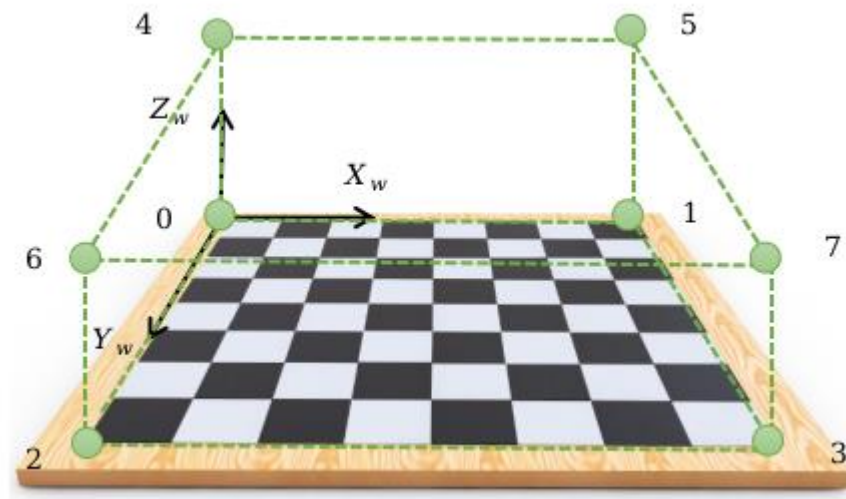


Figura 17. Cubo dibujado con realidad aumentada sobre un tablero de ajedrez.

El cubo se dibuja constantemente en cada imagen capturada siguiendo los siguientes pasos:

- 1) Se captura la imagen con el tablero de ajedrez a la vista.
- 2) Se obtienen pares puntos correspondientes 2D (distorsionados) y 3D.
Solo se usan los puntos 0, 1, 2 y 3 que se muestran en la Figura 17. Ya que con solo 4 pares de puntos correspondientes es suficiente para calcular la matriz G .
- 3) Se obtienen puntos 2D sin distorsión a partir de puntos 2D con distorsión.
- 4) Se calcula la matriz extrínseca (Q).
Esta matriz solo es válida para un instante de tiempo.
- 5) Se usa el modelo de cámara Pinhole no ideal para obtener los puntos 2D (distorsionados) de los vértices del cubo para poder colorearlos en la imagen (recordar que la imagen por lo general esta distorsionada).

Un ejemplo de los parámetros calculados para una cámara se muestra en la Figura 18.

```

USB PHY 2.0: USB CAMERA

PARAMETROS DE LA CAMARA

      K:
[[815.94925576 12.17008876 343.8569655]
 [ 0.      834.92166599 296.40028573]
 [ 0.      0.      1.    ]]

      q:
[[ 0.01009957]
 [ 0.00971951]
 [-0.00465224]
 [ 0.000763 ]
 [ 0.00058104]]

PARAMETROS DE CALIBRACION

Dimensiones tablero: (9, 6)
Tamaño cuadrado: 25.0

```

Figura 18. Parámetros de una cámara específica obtenidos en el proceso de calibración.

Un ejemplo del historial de pérdida de la función lagrangiana durante la optimización de los parámetros de la cámara se muestra en la Figura 19.



Figura 19. Historial de pérdida de la función objetivo.

En el proceso de calibración, al obtener los parámetros óptimos de la cámara, las matrices Q 's nos permiten visualizar como estaba orientado y posicionado el tablero de ajedrez con respecto al sistema de coordenadas de la cámara (c) para cada una de las matrices Q . Un ejemplo de esto se muestra en la Figura 20. El sistema de coordenadas de la izquierda es el sistema de coordenadas de la cámara (c) y los sistemas de coordenadas de la derecha son los sistemas de coordenadas del mundo (w) situados cada uno sobre el tablero de ajedrez correspondiente.

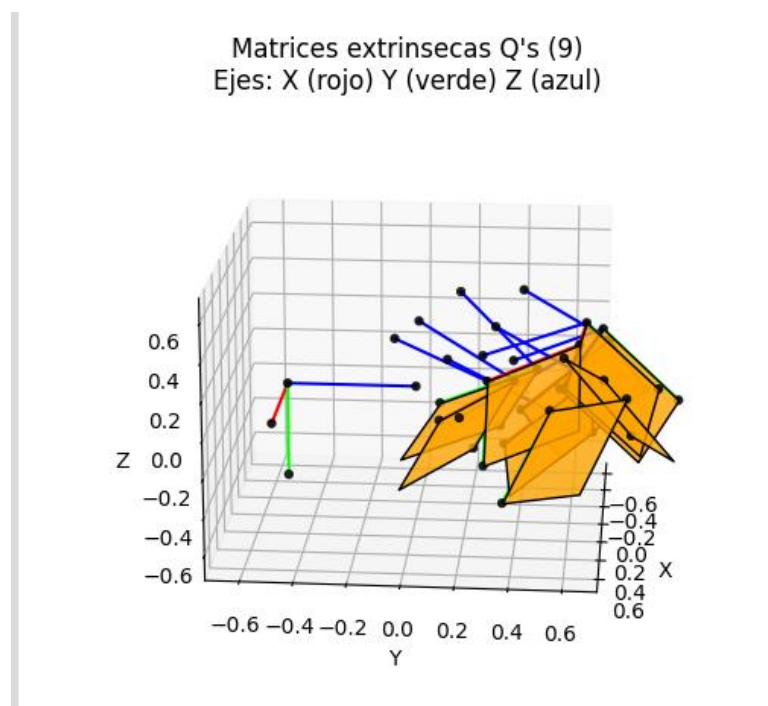


Figura 20. Visualización de las matrices extrínsecas (Q 's) obtenidas en el proceso de calibración.

En la aplicación de Python, el cubo se muestra como en la Figura 21.

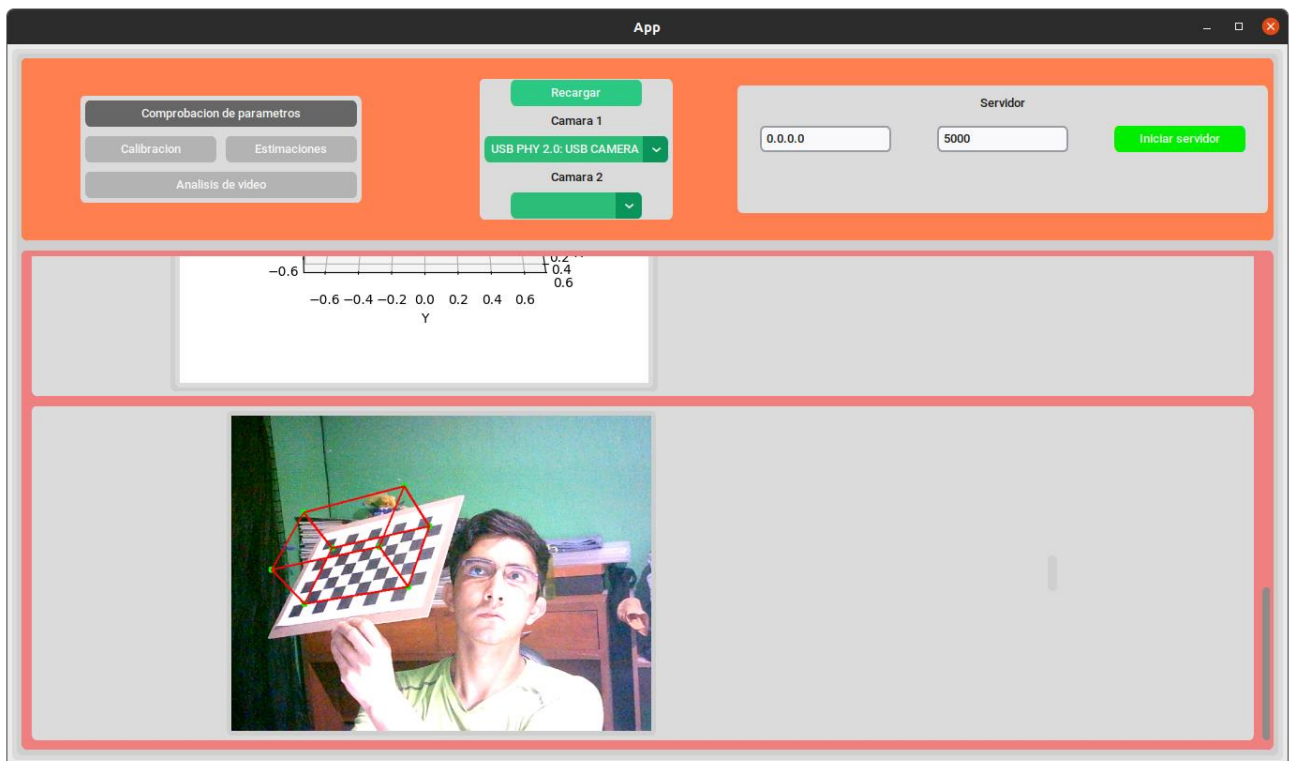


Figura 21. Comprobación de los parámetros de la cámara mediante realidad aumentada.

Una manera de tener los parámetros más óptimos para una cámara es utilizando un tablero de ajedrez grande, en el que se puedan obtener muchos pares de puntos correspondientes 2D y 3D, y calculando muchas matrices G , esto es, tomando muchas imágenes del tablero de ajedrez en diferentes posiciones y orientaciones. Esto hará que sea un poco más tardado el proceso de optimización, pero los parámetros obtenidos serán de mejor calidad.

5. Proyección de 2D a 3D

En este capítulo se hablará de los modelos de deep learning para la detección de puntos 2D característicos del cuerpo y de cómo estos son importantes en la estimación de puntos 3D para el análisis del movimiento corporal en 3D.

5.1. Modelos de deep learning para la detección de puntos 2D característicos del cuerpo

Para las estimaciones de puntos 3D característicos del cuerpo es necesario contar con modelos de deep learning que detecten esos puntos 3D pero sobre una imagen, es decir, su proyección sobre la imagen.

Los modelos de deep learning a utilizar son MediaPipe Hands [2] y MediaPipe Pose [3], ambos creados en el framework MediaPipe [13] desarrollado por investigadores de Google.

MediaPipe Hands funciona en tiempo real en dispositivos móviles e infiere 21 puntos clave de la mano en 2D y en 3D a partir de una imagen RGB. Cabe destacar que los puntos 3D están conformados por los puntos 2D y otro valor para cada punto que indica la profundidad relativa de dicho punto clave. La Figura 22 detalla los puntos clave que infiere el modelo MediaPipe Hands.

El modelo MediaPipe Hands tiene las siguientes características:

- Consta de dos modelos que funcionan juntos:
 - Detector de palmas (BlazePalm Detector)
Opera sobre la imagen completa y localiza las palmas a través de un cuadro delimitador.
 - Modelo de puntos clave de la mano (Hand landmark model)

Opera en el cuadro delimitador proporcionado por el detector de palma y devuelve puntos clave de la mano.

- Presenta una arquitectura de red de extracción de características (feature extractor) sobre la imagen de entrada, la cual comparten sus diferentes salidas.

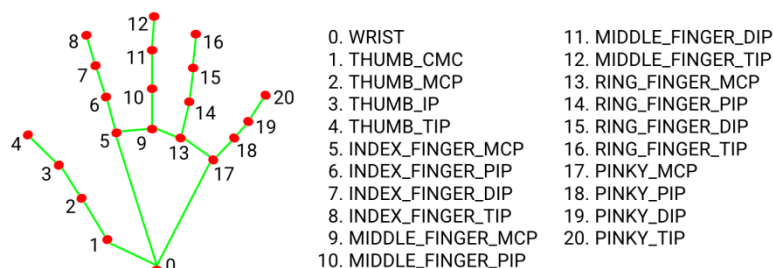


Figura 22. Puntos de referencia para el modelo MediaPipe Hands. [14]

MediaPipe Pose funciona en tiempo real en dispositivos móviles e infiere 33 puntos clave del cuerpo en 2D y en 3D, y además infiere una máscara de segmentación de fondo en todo el cuerpo a partir de una imagen RGB. Cabe destacar que los puntos 3D están conformados por los puntos 2D y otro valor para cada punto que indica la profundidad relativa de dicho punto clave. La Figura 23 detalla los puntos clave que infiere el modelo MediaPipe Pose.

El modelo MediaPipe Pose tiene las siguientes características:

- Presenta un seguimiento de la postura del cuerpo a lo largo del tiempo.
- Presenta una arquitectura de red encoder-decoder para predecir mapas de calor para todas las articulaciones presentes sobre la imagen y regresión para las coordenadas de cada punto clave.
- Es muy importante que la cabeza de la persona a detectar este visible en la imagen, ya que se cuenta con un detector facial rápido que detecta primero la cabeza y después de eso predice parámetros de alineación adicionales específicos de la persona, como lo son:
 - Punto medio entre las caderas de la persona.

- Tamaño del círculo que circunda a toda la persona.
- Inclinación.

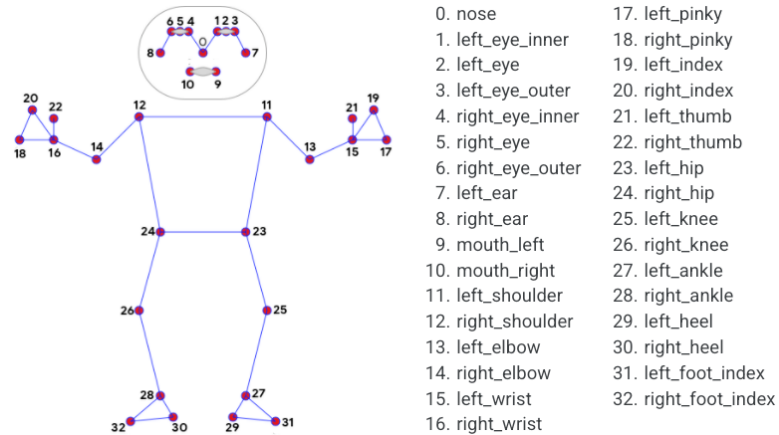


Figura 23. Puntos de referencia para el modelo MediaPipe Pose. [15]

Los modelos de MediaPipe se pueden instalar en Python mediante el siguiente comando:

```
pip install mediapipe
```

Para usar los modelos se debe declarar el siguiente código:

```
import mediapipe as mp

mp_hands=mp.solutions.hands
mp_pose=mp.solutions.pose
```

5.2. Estimaciones 3D de puntos característicos del cuerpo

Antes de continuar, es importante tener claro lo siguiente:

- Puntos 2D sin distorsión
 - Se usan en el modelo de cámara Pinhole ideal.
- Puntos 2D con distorsión
 - Se usan en el modelo de cámara Pinhole no ideal.

- Recordar que los puntos 2D con distorsión pueden pasar a ser puntos 2D sin distorsión mediante un proceso que elimina la distorsión.

Ahora bien, para realizar las estimaciones 3D se utilizan los modelos de deep learning MediaPipe Hands y MediaPipe Pose, así como el modelo de cámara Pinhole ideal y no ideal.

Para los modelos de detección de puntos 2D en una imagen, los puntos que se entregan por dichos modelos están distorsionados, esto debido a que las imágenes capturadas por la cámara tienen distorsión, por tanto, corresponden al modelo no ideal.

Para poder tener estimaciones 3D por lo menos se necesitan 2 cámaras calibradas y se necesita definir un sistema de coordenadas del mundo (w) en donde estén representados los puntos 3D. La matriz extrínseca (Q) es la que define como se encuentra orientado y posicionado dicho sistema con respecto al sistema de coordenadas de la cámara (c). La equ. 111 muestra como calcular la matriz extrínseca (Q). La posición y orientación del sistema de coordenadas del mundo (w) es muy importante al momento de interpretar las estimaciones de puntos 3D.

El cálculo de la matriz extrínseca (Q) se puede automatizar mediante pares de puntos correspondientes 2D y 3D a partir de un marcador ArUco, tal y como se muestra en la Figura 24. Se utiliza el marcador ArUco debido a que es muy fácil y rápido de detectar en una imagen. De este marcador ArUco se pueden obtener 4 puntos sobre la imagen, los cuales son suficientes para poder calcular la matriz extrínseca (Q). Lo que se necesita conocer del marcador ArUco es lo siguiente:

- Longitud de un lado del cuadrado (en milímetros)
- ID del marcador ArUco
- Tipo de diccionario

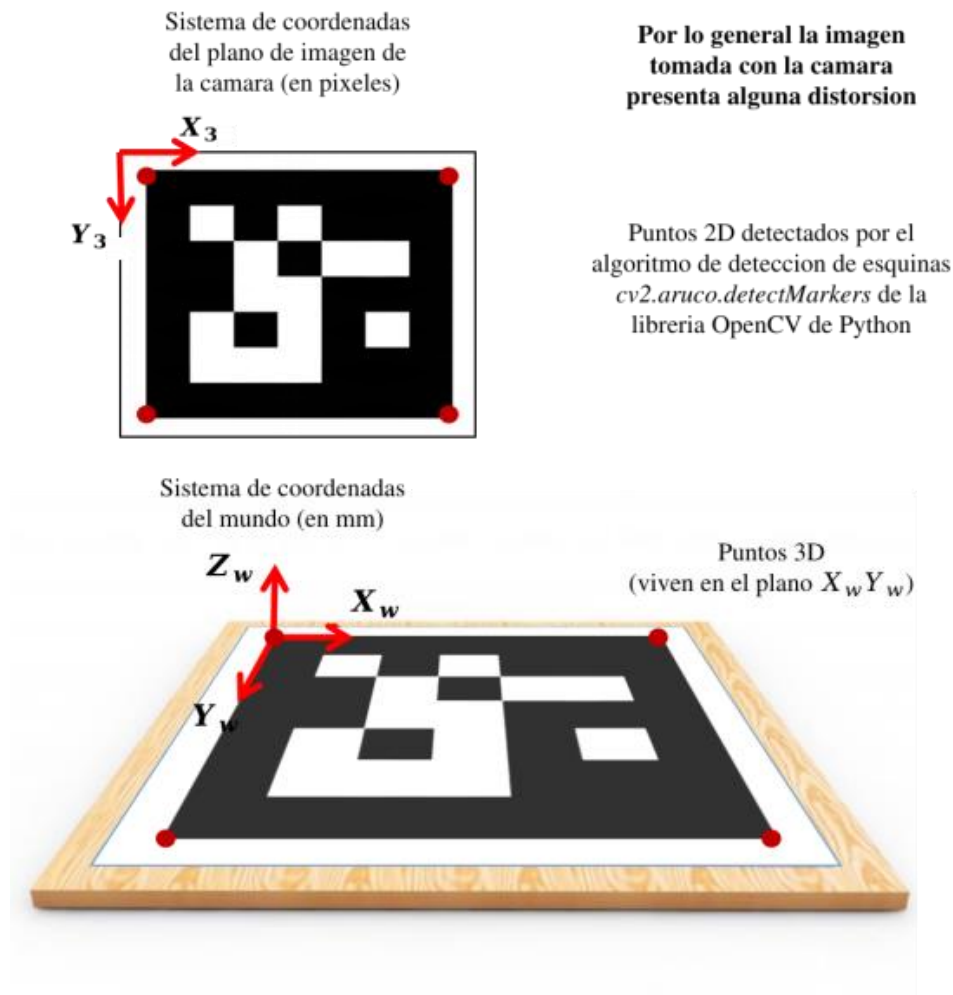


Figura 24. Obtención de pares de puntos correspondientes 2D y 3D a partir de un marcador ArUco.

Al calcular la matriz extrínseca (Q) en la aplicación de Python, se sugiere que la colocación del marcador ArUco sea como se muestra en la Figura 25. Esto es debido a que internamente se realizan algunas transformaciones entre sistemas de coordenadas, quedando como sistema de referencia un sistema más cómodo y práctico para utilizar. Además, este nuevo sistema coincide con el sistema de coordenadas en donde se grafican los puntos 3D en la aplicación de Python.

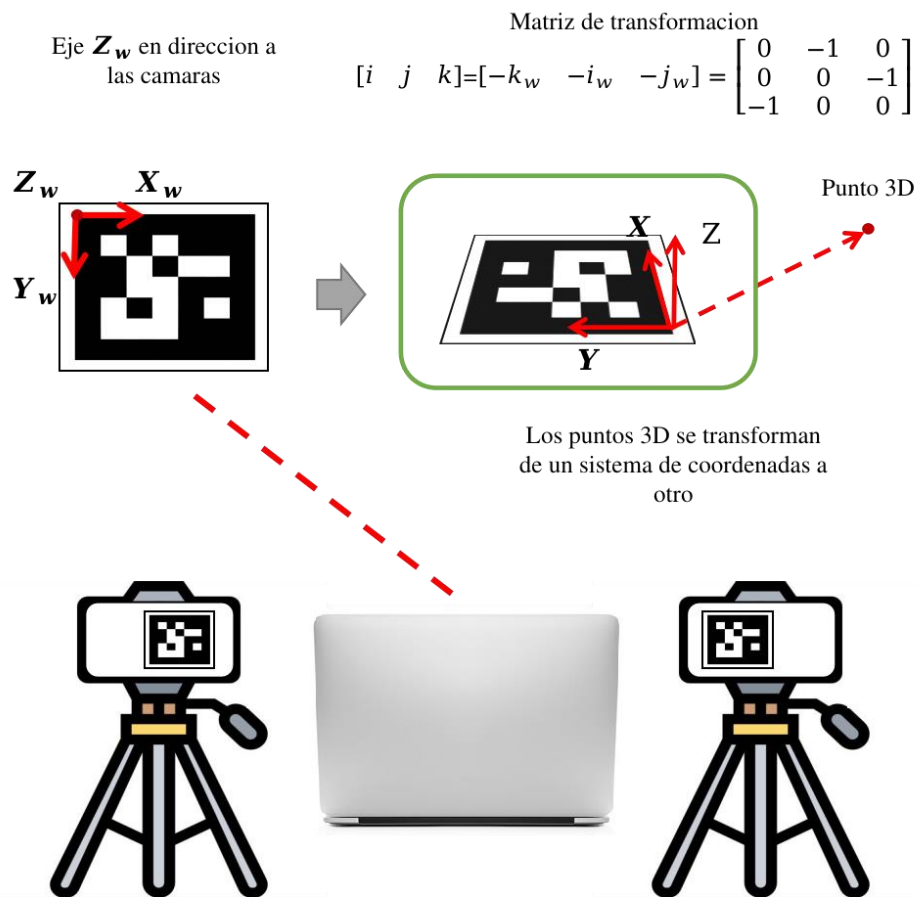


Figura 25. Colocación del marcador ArUco al momento de calcular la matriz extrínseca (Q).

En la Figura 25, el sistema de coordenadas con el que se trabaja finalmente es el que se encuentra encerrado en el recuadro de color verde.

El proceso que se sigue para obtener un solo punto 3D a partir de 2 cámaras calibradas es el siguiente:

- 1) Se calculan las matrices extrínsecas para cada cámara utilizando la equ. 111. Se tendría lo siguiente:

$Q^{(1)} = h(G^{(1)}, K^{(1)})$	(112)
$Q^{(2)} = h(G^{(2)}, K^{(2)})$	(113)

Donde:

- $Q^{(p)}$: Corresponde a la matriz extrínseca de la cámara p
- $G^{(p)}$: Corresponde a la matriz que se obtiene mediante pares de puntos correspondientes 2D y 3D de la cámara p
- $K^{(p)}$: Corresponde a la matriz intrínseca de la cámara p

- 2) Dados dos puntos 2D distorsionados provenientes de cámaras diferentes pero del mismo punto 3D, se utiliza la equ. 110 para encontrar sus correspondientes puntos 2D sin distorsión. Se tendría lo siguiente:

$v_{3p}^{(1)} = \text{undistorted}(v_{3pd}^{(1)}, K^{(1)}, q^{(1)})$	(114)
$v_{3p}^{(2)} = \text{undistorted}(v_{3pd}^{(2)}, K^{(2)}, q^{(2)})$	(115)

Donde:

- $v_{3pd}^{(p)}$: Corresponde al punto 2D con distorsión de la cámara p
- $v_{3p}^{(p)}$: Corresponde al punto 2D sin distorsión de la cámara p
- $K^{(p)}$: Corresponde a la matriz intrínseca de la cámara p
- $q^{(p)}$: Corresponde a los parámetros de distorsión de la cámara p

Para comprender mejor lo anterior consultar la Figura 26.

- 3) Se define lo siguiente:

$P = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} = KQ$	(116)
--	--------

A la matriz P se le conoce como matriz de proyección.

Quedando la equ. 46 de la siguiente manera:

$v_{cz}m = PM$ $v_{cz} \begin{bmatrix} v_{3px} \\ v_{3py} \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} v_{wx} \\ v_{wy} \\ v_{wz} \\ 1 \end{bmatrix}$	(117)
---	--------

- 4) Se desglosan las ecuaciones para el punto 3D para cada cámara utilizando la equ. 117 y se construye un sistema de ecuaciones. Se tendría lo siguiente:

$v_{cz}^{(1)}m^{(1)} = P^{(1)}M$ $v_{cz}^{(1)} \begin{bmatrix} v_{3px}^{(1)} \\ v_{3py}^{(1)} \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11}^{(1)} & p_{12}^{(1)} & p_{13}^{(1)} & p_{14}^{(1)} \\ p_{21}^{(1)} & p_{22}^{(1)} & p_{23}^{(1)} & p_{24}^{(1)} \\ p_{31}^{(1)} & p_{32}^{(1)} & p_{33}^{(1)} & p_{34}^{(1)} \end{bmatrix} \begin{bmatrix} v_{wx} \\ v_{wy} \\ v_{wz} \\ 1 \end{bmatrix}$	(118)
$v_{cz}^{(2)}m^{(2)} = P^{(2)}M$ $v_{cz}^{(2)} \begin{bmatrix} v_{3px}^{(2)} \\ v_{3py}^{(2)} \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11}^{(2)} & p_{12}^{(2)} & p_{13}^{(2)} & p_{14}^{(2)} \\ p_{21}^{(2)} & p_{22}^{(2)} & p_{23}^{(2)} & p_{24}^{(2)} \\ p_{31}^{(2)} & p_{32}^{(2)} & p_{33}^{(2)} & p_{34}^{(2)} \end{bmatrix} \begin{bmatrix} v_{wx} \\ v_{wy} \\ v_{wz} \\ 1 \end{bmatrix}$	(119)

Desglosando para el punto 3D, su forma matricial estaría representada de la siguiente manera:

$\begin{bmatrix} p_{31}^{(1)}v_{3px}^{(1)} - p_{11}^{(1)} & p_{32}^{(1)}v_{3px}^{(1)} - p_{12}^{(1)} & p_{33}^{(1)}v_{3px}^{(1)} - p_{13}^{(1)} \\ p_{31}^{(1)}v_{3py}^{(1)} - p_{21}^{(1)} & p_{32}^{(1)}v_{3py}^{(1)} - p_{22}^{(1)} & p_{33}^{(1)}v_{3py}^{(1)} - p_{23}^{(1)} \\ p_{31}^{(2)}v_{3px}^{(2)} - p_{11}^{(2)} & p_{32}^{(2)}v_{3px}^{(2)} - p_{12}^{(2)} & p_{33}^{(2)}v_{3px}^{(2)} - p_{13}^{(2)} \\ p_{31}^{(2)}v_{3py}^{(2)} - p_{21}^{(2)} & p_{32}^{(2)}v_{3py}^{(2)} - p_{22}^{(2)} & p_{33}^{(2)}v_{3py}^{(2)} - p_{23}^{(2)} \end{bmatrix} \begin{bmatrix} v_{wx} \\ v_{wy} \\ v_{wz} \end{bmatrix}$ $= \begin{bmatrix} p_{14}^{(1)} - p_{34}^{(1)}v_{3px}^{(1)} \\ p_{24}^{(1)} - p_{34}^{(1)}v_{3py}^{(1)} \\ p_{14}^{(2)} - p_{34}^{(2)}v_{3px}^{(2)} \\ p_{24}^{(2)} - p_{34}^{(2)}v_{3py}^{(2)} \end{bmatrix}$	(120)
--	--------

La equ. 120 podría expresarse de la forma $Ax = b$.

Es importante observar que se necesitan por lo menos 2 cámaras calibradas para que el sistema de ecuaciones sea candidato a tener una solución.

- 5) La solución para el sistema de ecuaciones es la siguiente (para más información consultar el anexo 1):

$x = (A^T A)^{-1} A^T b = \begin{bmatrix} v_{wx} \\ v_{wy} \\ v_{wz} \end{bmatrix}$	(121)
---	--------

La equ. 121 entrega el punto 3D expresado en el sistema de coordenadas del mundo (w).

- 6) Posterior al cálculo del punto 3D, el cual se encuentra expresado en el sistema de coordenadas del mundo (w), se transforma para que este expresado en el nuevo sistema de coordenadas, tal y como se muestra en la Figura 25. Además, se hace una conversión de milímetros a metros para el punto 3D.
- 7) Para estimar más puntos 3D se deben repetir los pasos del 2 al 6, tomando en cuenta que las cámaras no se han movido y de que no es necesario calcular de nuevo las matrices extrínsecas de cada cámara.

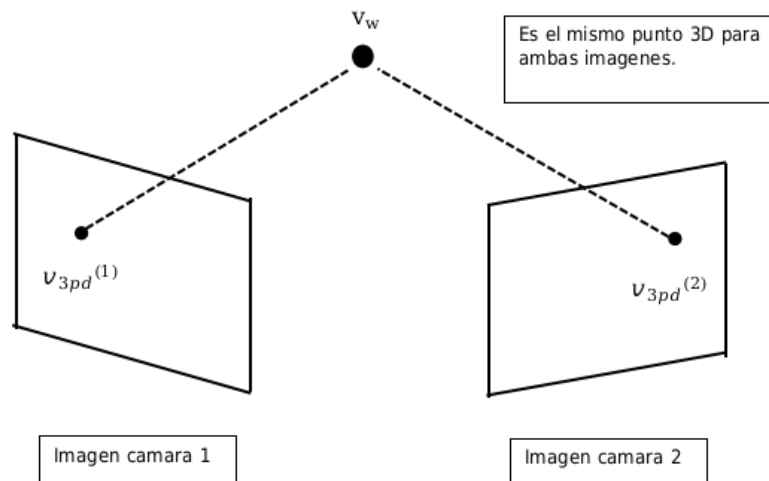


Figura 26. Proyección de un punto 3D sobre los planos de imagen de las cámaras.

Es importante mencionar que una vez iniciadas las estimaciones de los puntos 3D las cámaras no se deben de mover. Si por algún motivo las cámaras se mueven, la matriz extrínseca (Q) previamente calculada para cada cámara ya no será la correcta, ya que la posición y orientación del sistema de coordenadas del mundo (w) con respecto al sistema de coordenadas de cada cámara (c) habrá cambiado, y la matriz extrínseca (Q) precisamente expresa esa relación. Además, todo esto traerá estimaciones de puntos 3D incorrectas, lo cual no es deseable. Por tanto, lo que queda por hacer es repetir el procedimiento para iniciar las estimaciones de puntos 3D, asegurando no mover las cámaras en este proceso.

Criterios para que las estimaciones de puntos 3D sean precisos:

- Los parámetros de la cámara deben ser los más precisos para cada cámara.
- Los puntos 2D en cada cámara provenientes del mismo punto 3D deben ser si o si correspondientes.

La Figura 27 muestra como son las estimaciones de puntos 3D en la aplicación de Python.

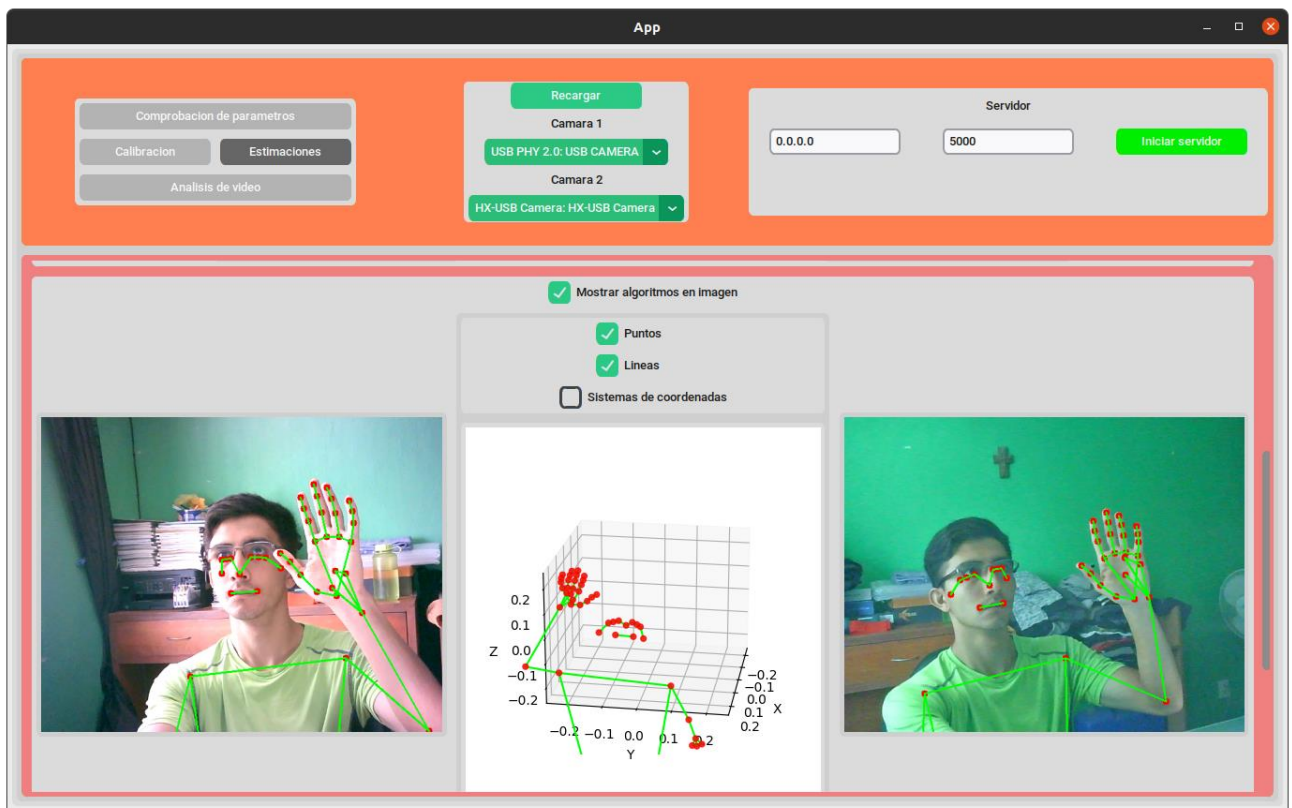


Figura 27. Estimaciones de puntos 3D.

La Figura 28 muestra los modelos de detección de puntos 2D que se pueden utilizar y los parámetros de las cámaras seleccionadas. Es importante recordar que se necesitan por lo menos 2 cámaras calibradas para llevar a cabo la estimación de puntos 3D.

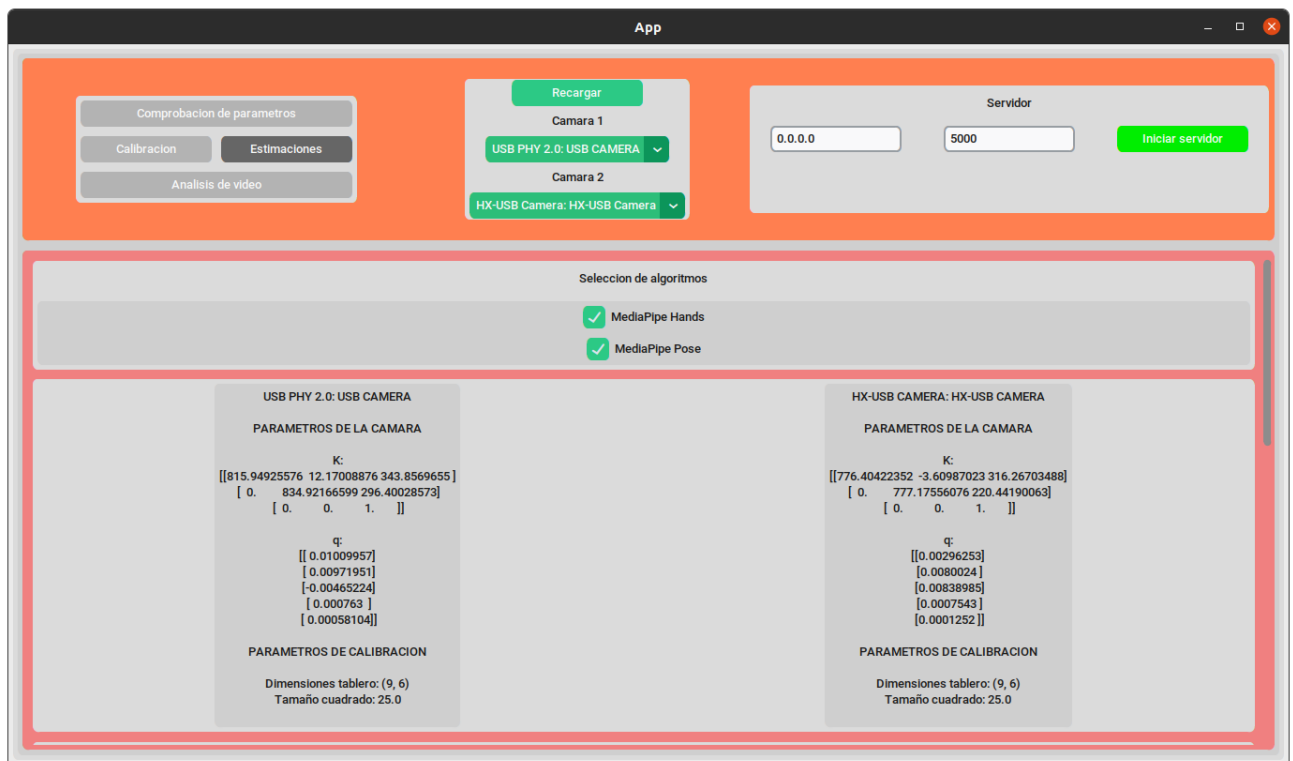


Figura 28. Modelos de detección de puntos 2D seleccionables.

La Figura 29 muestra el apartado para calcular las matrices extrínsecas de las cámaras seleccionadas. Para más información consultar la Figura 25 sobre el acomodo del marcador ArUco para calcular las matrices extrínsecas.

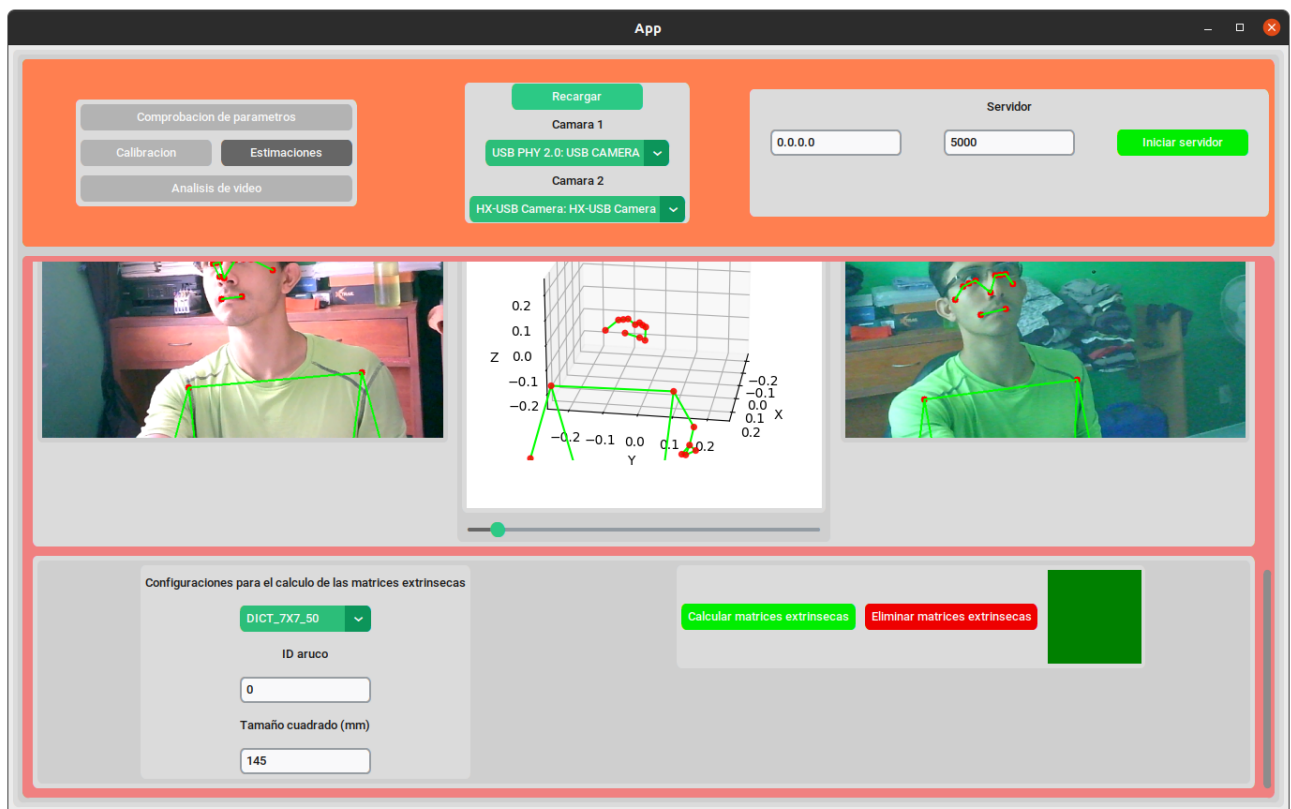


Figura 29. Calculo de matrices extrínsecas.

5.3. Análisis del movimiento corporal en 3D

A partir de las estimaciones de puntos 3D de puntos característicos del cuerpo es posible analizar la estructura corporal de una persona.

El análisis de movimiento que se realiza en la aplicación de Python es en base a un video pregrabado. Para llevar a cabo el análisis se deben seguir los siguientes pasos:

- 1) Utilizando dos cámaras calibradas se graba un video, para cada cámara, de la persona haciendo alguna actividad física o de algún movimiento que se desea analizar más detalladamente. Este apartado se muestra en la Figura 30.
- 2) Posterior a la grabación de los videos, en otro apartado se mostrarán las imágenes de cada video en un instante de tiempo específico, en el cual se podrán recorrer los

videos imagen por imagen, asegurando que las imágenes correspondan al mismo instante de tiempo. Este apartado se muestra en la Figura 31.

- 3) Mediante las imágenes de los videos es posible realizar estimaciones de puntos 3D para analizar la estructura corporal en 3D de la persona. Para ello se deben seleccionar los modelos de detección de puntos 2D a utilizar y se deben calcular las matrices extrínsecas de las cámaras calibradas seleccionadas.
- 4) Teniendo las estimaciones de puntos 3D es posible realizar cierto análisis con respecto al movimiento de la persona. El análisis que se puede hacer es con respecto a los ángulos articulares de la persona, los cuales se pueden visualizar a lo largo del tiempo. Este apartado se muestra en la Figura 32.

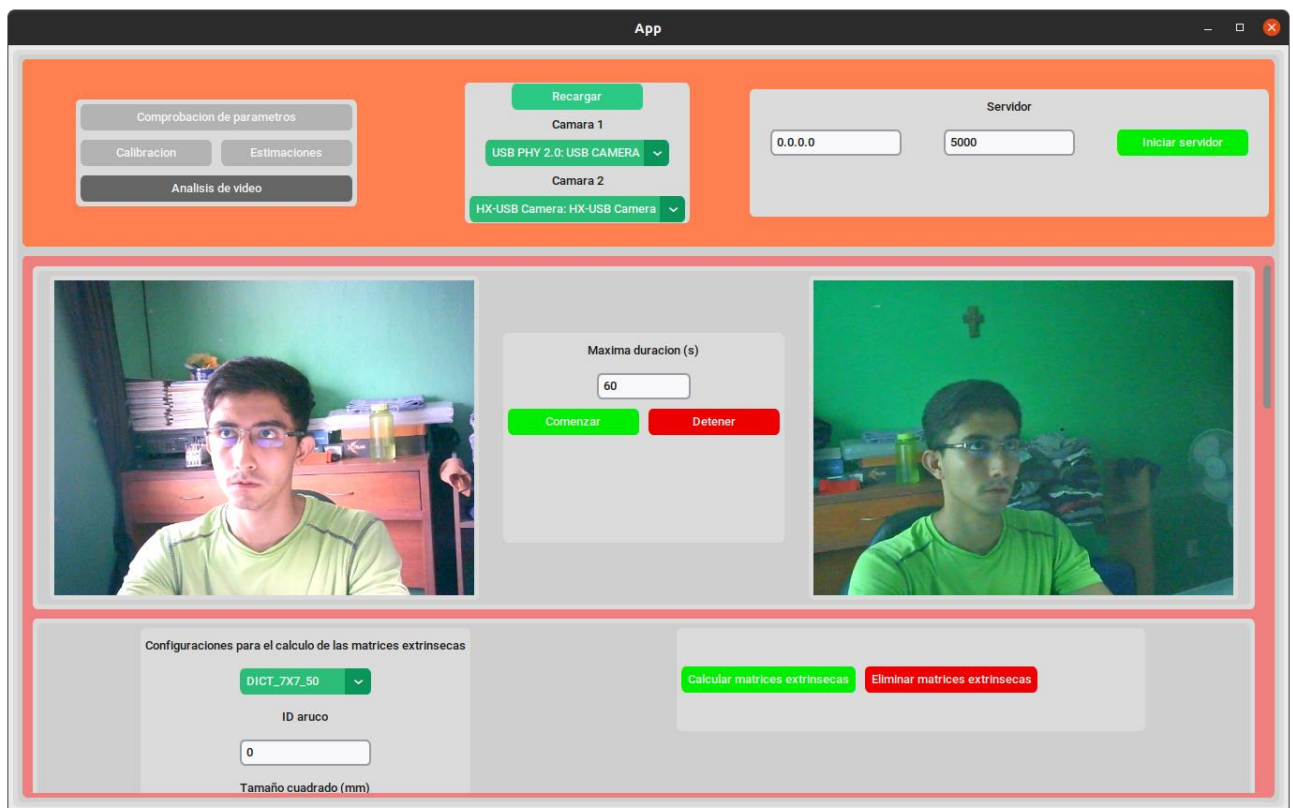


Figura 30. Captura de videos.

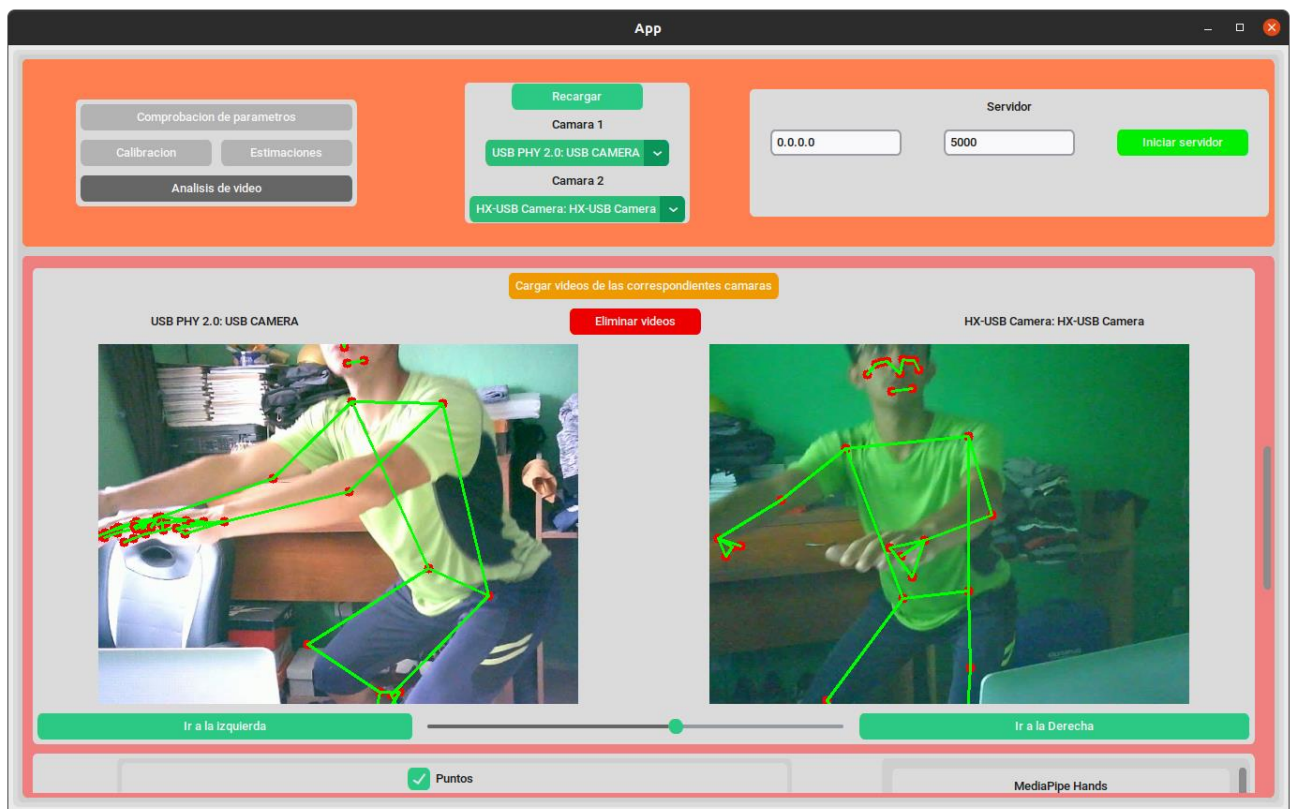


Figura 31. Visualización de videos imagen por imagen.

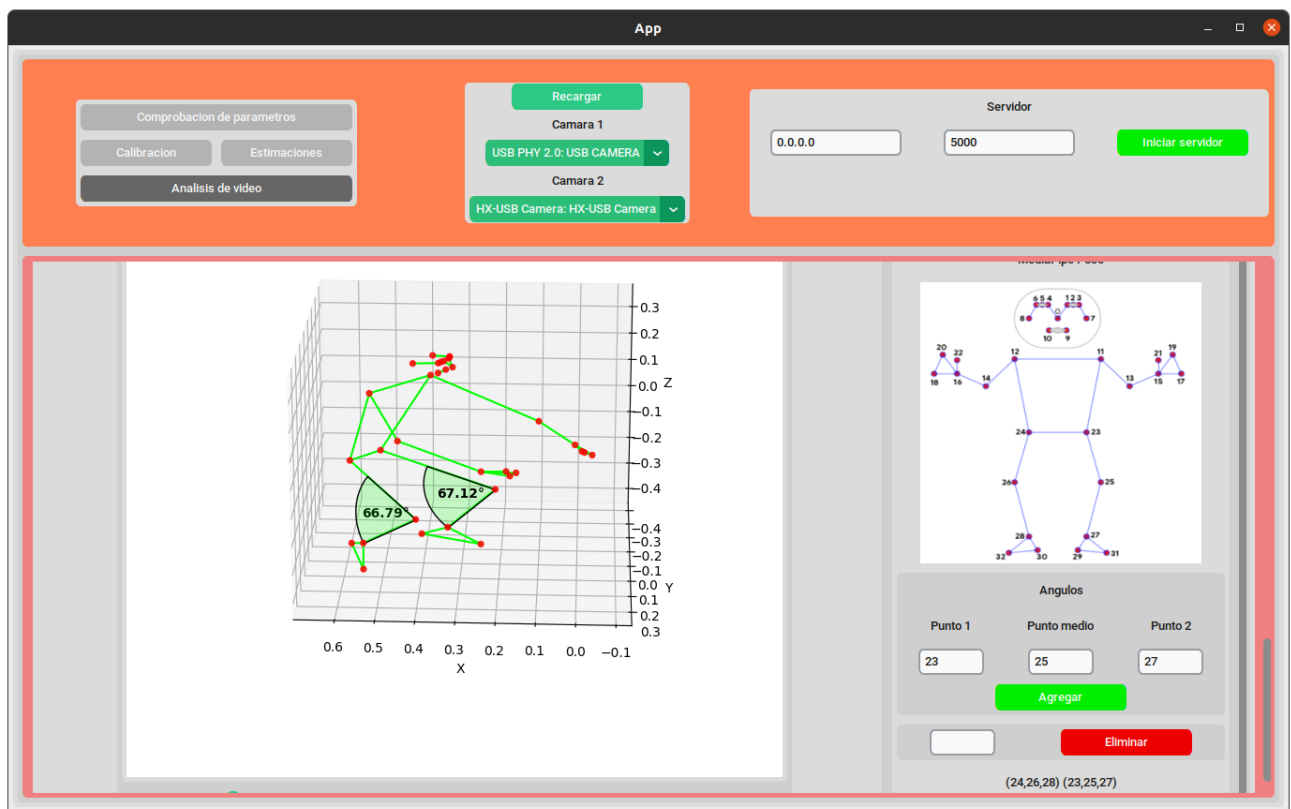


Figura 32. Visualización de ángulos articulares de una persona.

La importancia de visualizar los ángulos articulares de una persona a lo largo del tiempo es debido a que muchos ejercicios requieren hacerse de manera correcta, y este análisis permite corregir la ejecución del ejercicio y además permite ver posibles anomalías en el movimiento de una persona.

Para visualizar un ángulo articular se debe elegir un modelo y 3 puntos de dicho modelo. La Figura 33 muestra más detalle de cómo se eligen estos puntos.

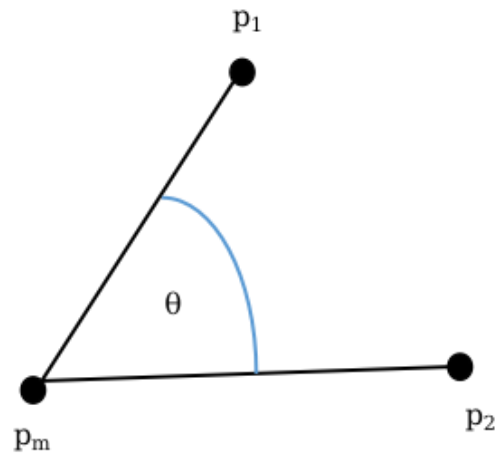


Figura 33. Ángulo formado por 3 puntos.

El análisis se realiza con las mismas cámaras con las que fueron grabados los videos y se realiza imagen por imagen. Los videos grabados se eliminan automáticamente después de volver a grabar otros videos, o también se pueden eliminar manualmente.

6. Aplicación de Python y Unity

En este capítulo se explicará a grandes rasgos el trabajo realizado en la aplicación de Python y en la aplicación de Unity y como esté último se comunica con el sistema de visión estéreo para su posible uso en la rehabilitación física mediante juegos interactivos 3D controlados mediante el movimiento corporal.

6.1. Aplicación de Python

Anteriormente se han mostrado fragmentos de la aplicación de Python pero no se ha mostrado en grandes rasgos las partes que la componen. Recordar que tal aplicación esta lanzada como un servicio mediante Docker-Compose.

La aplicación consta de 3 partes importantes:

- Navegación entre apartados
 - Apartado de comprobación de parámetros de la cámara
 - Apartado de calibración de la cámara
 - Apartado de estimaciones de puntos 3D
 - Apartado de análisis en video del movimiento corporal en 3D
- Selección de cámaras

Aquí se seleccionan las cámaras a utilizar. Dado a que se está trabajando con visión estéreo, es posible seleccionar 2 cámaras. En las listas desplegables se muestran todas las cámaras USB conectadas a la computadora, así como la cámara web en caso de estar trabajando en una computadora portátil.
- Servidor

Aquí se inicia el servidor. El servidor se cierra en automático al salir de la aplicación.

La Figura 34 muestra la interfaz de la aplicación de Python, en la cual se puede ver cómo están distribuidos las diferentes partes que la componen.

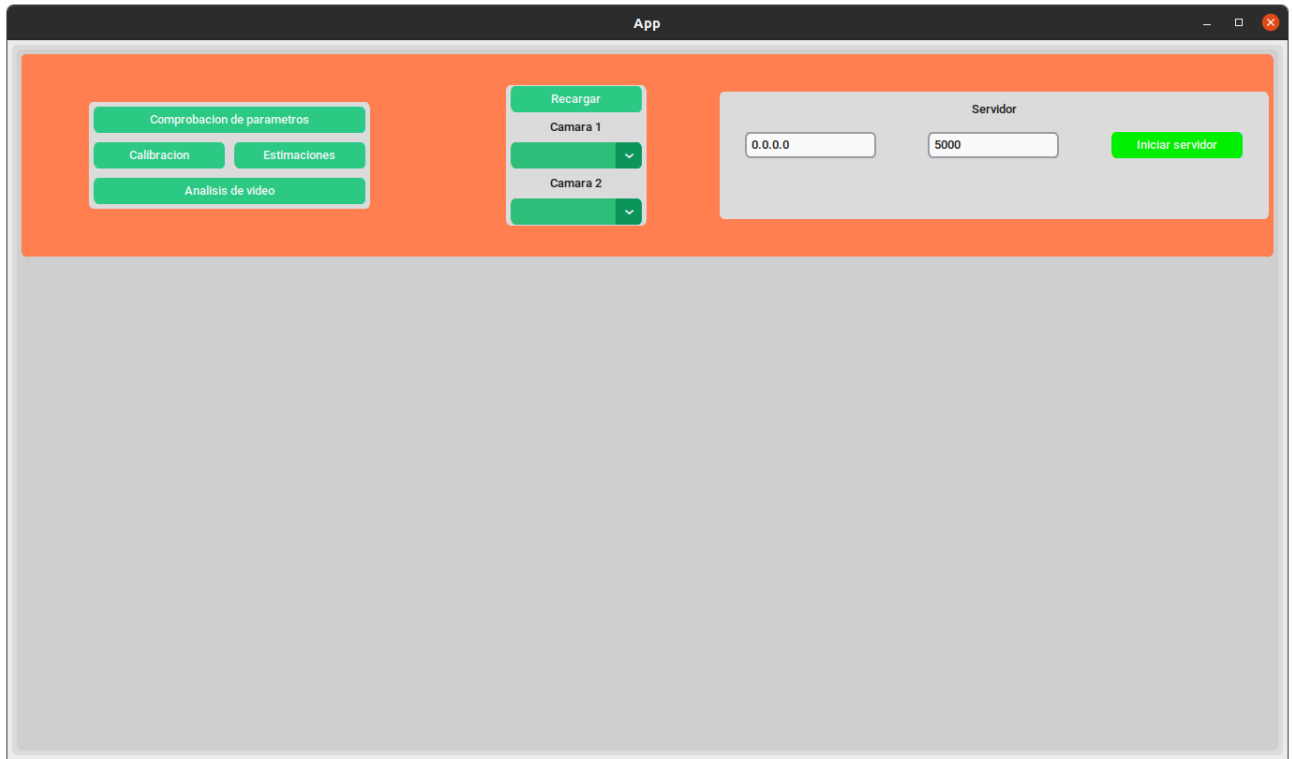


Figura 34. Vista principal de la aplicación de Python.

La aplicación ejecuta 2 hilos (threads) de principio a fin que se encargan del control de las cámaras seleccionadas, permitiendo obtener frame por frame de lo que captura cada una de las cámaras en cada instante de tiempo. Es muy importante que el código de estos hilos se ejecute de manera independiente y concurrente dentro del proceso principal, lo cual permite tener un mejor rendimiento en la aplicación.

6.2. Aplicación de Unity

La aplicación de Unity no es como tal una aplicación que tenga muchas funcionalidades. Su objetivo principal es servir de base para trabajos posteriores en donde se requiera utilizar

Unity para interactuar con juegos 3D mediante el movimiento corporal de una persona. Todo esto es posible haciendo uso de todo lo que se ha estado viendo hasta el momento.

La aplicación consta por el momento de 2 modelos de mano, una para la mano izquierda y otro para la mano derecha, los cuales se ajustan muy bien al modelo MediaPipe Hands, esto debido a que las manos tienen el mismo número de articulaciones que los puntos detectados por dicho modelo, lo cual es muy importante.

La Figura 35 muestra la aplicación de Unity desde el editor, en donde se pueden observar los componentes que la componen. Entre estos se destacan los siguientes:

- SocketMessageCanvas

Es un componente visual y es el que está situado en la parte superior, el cual contiene los diferentes botones e inputs para conectar y desconectar al cliente. Además, en este componente se muestran todos los mensajes relacionados con la comunicación entre el cliente y el servidor.

- CameraHands

Es un componente de cámara y se encarga de mostrar los modelos 3D en escena.

- Plane

Es un componente que sirve para delimitar el suelo.

- Cube

Es un componente que solo sirve de referencia para saber dónde está colocado el origen del sistema de coordenadas de Unity.

- LeftHand

Es el modelo 3D para la mano izquierda, el cual cuenta con 21 articulaciones.

- RightHand

Es el modelo 3D para la mano derecha, el cual cuenta con 21 articulaciones.

- SocketClient

Es un componente vacío que tiene asociado un script. Este script ejecuta un hilo (thread) que se encarga del control de la conexión y comunicación entre el cliente y el servidor, permitiendo ejecutar código concurrentemente con el hilo principal, lo que mejora la eficiencia del programa.

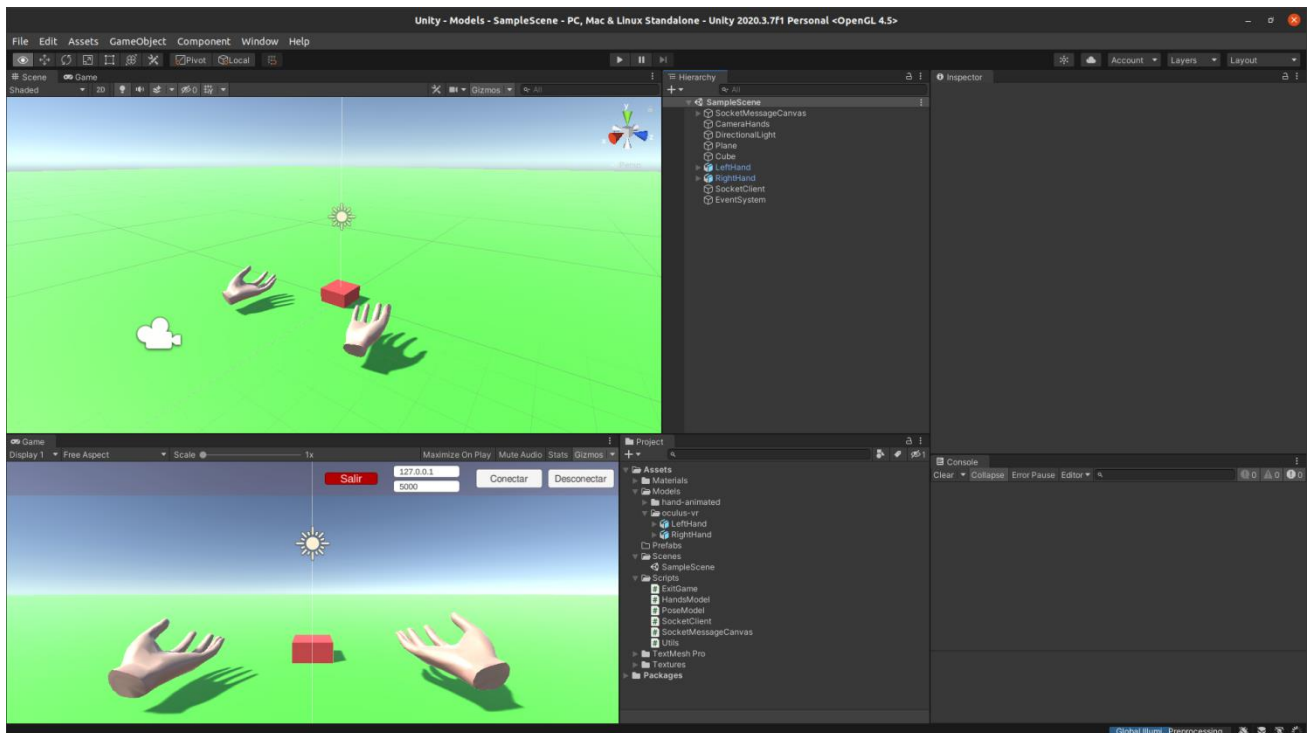


Figura 35. Vista desde el editor de la aplicación de Unity.

Se cuenta con un apartado en la parte superior de la aplicación con opciones para conectar y desconectar al cliente. El cliente no se conecta en automático con el servidor al iniciar la aplicación, es algo que se tiene que hacer a mano si se desea recibir datos por parte del servidor.

Si el cliente logra conectarse al servidor, la única manera para que se reciban datos es que en la aplicación de Python se esté en el apartado de estimaciones y se tenga activado el modelo

de MediaPipe Hands, ya que por el momento son los datos necesarios para mover los modelos de manos.

6.3. Comunicación y envío de datos entre la aplicación de Python y la aplicación de Unity

Las estimaciones de puntos 3D de puntos característicos del cuerpo no solo pueden ser usados para el análisis del movimiento corporal en 3D, sino que también pueden ser utilizados por otra aplicación para llevar a cabo otro tipo de tareas. Tal es el caso de Unity, en el que se pueden utilizar las estimaciones de puntos 3D para una gran variedad de tareas. Para ello es necesario que exista una comunicación entre la aplicación de Python y la aplicación de Unity, lo cual se puede lograr mediante una comunicación vía Socket TCP/IP.

Se optó por una arquitectura cliente-servidor como se muestra en la Figura 36. El motivo de usar el protocolo TCP/IP es debido a que garantiza que toda la transmisión de datos se haga sin errores y además garantiza el orden, lo cual es muy importante cuando se habla de transmisión de datos que se generan a lo largo del tiempo. Este protocolo, a comparación con el protocolo UDP, requiere establecer una conexión entre el cliente y el servidor para poder enviar datos, ya que si no existe no es posible enviar datos de un extremo a otro.

Al ejecutar la aplicación de Python mediante Docker-Compose se mapea el puerto 5000 de la computadora local con el puerto 5000 del contenedor, ya que el puerto 5000 será el puerto por defecto en donde estará escuchando el servidor y por donde se estará llevando toda la comunicación. Esto es importante para que la aplicación de Unity pueda establecer una conexión con el servidor, el cual se encuentra lanzado como un servicio mediante Docker-Compose. De lo contrario no habría manera de establecer una conexión entre ambas aplicaciones. La Figura 37 muestra cómo se deben configurar ambas aplicaciones.

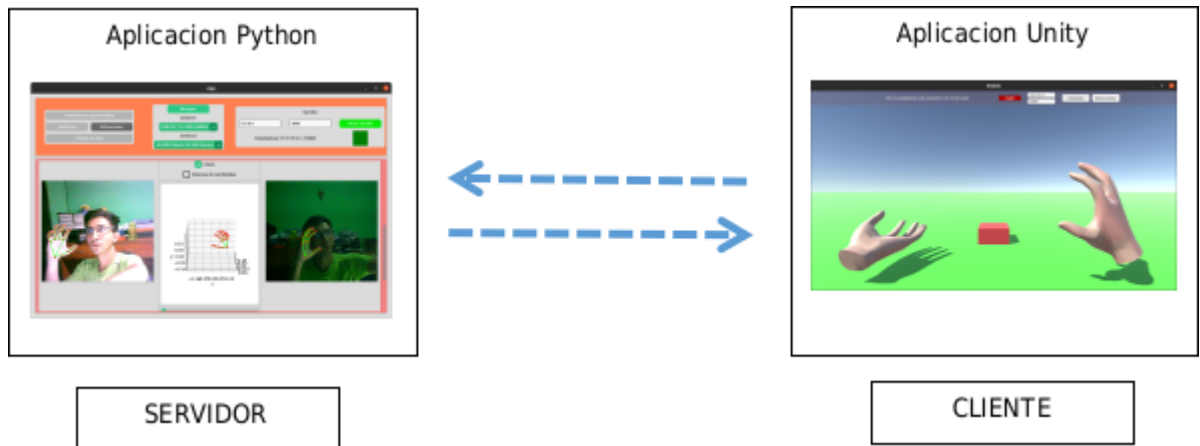


Figura 36. Arquitectura cliente-servidor entre aplicaciones.

Pasos para establecer la conexión entre aplicaciones:

- 1) Iniciar servidor en la aplicación de Python
- 2) Conectar con el servidor desde la aplicación de Unity

Una vez establecida la conexión, lo que sucede es lo siguiente:

- 1) El cliente envía un valor al servidor para solicitar todos los datos necesarios para trabajar en Unity. Después de esto queda a la espera de los datos.
- 2) El servidor recibe el valor por parte del cliente y envía los datos necesarios a Unity (si es que están disponibles). Después de esto queda a la escucha de nuevo de lo que envíe el cliente.

Los datos que se envían del servidor al cliente se envían en un solo envío y tienen una estructura específica. La estructura de los datos es la siguiente:

- Numero de modelos seleccionados (4 bytes)
Por modelo seleccionado se envía lo siguiente:
 - Nombre modelo (20 bytes)
 - Número de puntos (4 bytes)
 - Numero de partes (4 bytes)

Una parte puede ser “Left”, “Right” o “All”, haciendo referencia a modelos que detectan puntos 2D de parte izquierda y derecha, o de todo en general.

Por parte se envía lo siguiente:

- ◆ Nombre de parte (20 bytes)
- ◆ Puntos 3D ((número de puntos del modelo)*3*4 bytes)
 Los puntos 3D se transforman antes de enviarse para que se encuentren expresados con respecto al sistema de coordenadas de Unity.
- ◆ Ángulos de Euler para cada punto 3D ((número de puntos del modelo)*3*4 bytes)
 Los ángulos de Euler se pueden determinar a partir de los puntos 3D y son 3 ángulos por punto 3D. Los ángulos se envían en grados (ya que Unity trabaja con grados). Para más información consultar el anexo 4.

Toda la información que se envía al cliente se acumula de acorde a la estructura anterior antes de enviarse. Es importante mencionar que en la aplicación de Python se debe estar en el apartado de estimaciones para que se puedan enviar datos al cliente, ya que aquí es donde se hacen estimaciones de puntos 3D en tiempo real. Los datos que se envían dependen de los modelos seleccionados en el apartado de estimaciones.

En Unity, el cliente no lee todos los datos en una sola lectura, ya que no se sabe con certeza el número de bytes que se necesitan leer, pero si sabe la estructura que llevan los datos, por tanto, el cliente lee por partes los datos recibidos de igual forma como fueron estructurados por el servidor antes de enviarlos. Esto permite recibir datos con longitudes variadas pero que respetan una misma estructura, la cual la determina el servidor.

The image shows a graphical user interface for configuring a server and client connection. It is divided into two main sections: 'SERVIDOR' (Server) and 'CLIENTE' (Client).

SERVIDOR Section:

- Contains a label 'Servidor'.
- Has two input fields: the first contains '0.0.0.0' and the second contains '5000'.
- Includes a green button labeled 'Iniciar servidor'.

CLIENTE Section:

- Contains a red button labeled 'Salir'.
- Has two input fields: the first contains '127.0.0.1' and the second contains '5000'.
- Includes two buttons: 'Conectar' and 'Desconectar'.

Figura 37. Configuraciones para la conexión entre servidor y cliente.

6.4. Interacción con modelos 3D en Unity a partir de la estimación 3D de puntos característicos del cuerpo

Para poder mover modelos 3D en Unity es importante que los modelos 3D tengan el mismo número de articulaciones y que coincidan en posición con los puntos detectados por los modelos de deep learning. Esto hará que los modelos 3D se ajusten mejor a los movimientos corporales de la persona. En pocas palabras, el modelo 3D debe ser compatible con el modelo de deep learning.

Existe la posibilidad de crear modelos 3D en Blender [16] de tal manera que tengan la misma distribución de articulaciones que algún modelo de deep learning específico. Esto permitiría aumentar las posibilidades de compatibilidad entre modelo 3D y modelo de deep learning.

Los únicos modelos 3D que se utilizan son los modelos 3D para las manos, cuyos datos provienen del modelo MediaPipe Hands. Para el modelo MediaPipe Pose no fue posible encontrar un modelo 3D que tuviera la misma distribución de articulaciones que el modelo de deep learning. Aun así, con los modelos 3D para las manos es suficiente para mostrar que es

posible controlar dichos modelos mediante los datos recibidos por el servidor desde la aplicación de Python.

Con la información que se recibe es suficiente para poder mover modelos 3D. De los datos más importantes que se reciben por cada modelo de deep learning son los siguientes:

- Puntos 3D (en metros)
Se encuentran expresados en el sistema de coordenadas de Unity.
- Ángulos de Euler por cada punto 3D (en grados)
Recordar que los ángulos de Euler se calculan a partir del sistema de coordenadas que se define en el punto 3D.

Los pasos que se siguen para preparar un modelo 3D y poderlo mover son los siguientes:

- 1) Es importante asegurar que el modelo 3D sea compatible con el modelo de deep learning correspondiente. Por lo general primero es el modelo de deep learning y después se elige un modelo 3D que mejor se acople. Existen muchos modelos 3D gratuitos que se pueden descargar de internet, o si se desea es posible crear un modelo 3D desde cero con Blender.
- 2) Una vez cargado el modelo en escena, se identifican las articulaciones de interés de dicho modelo y se les cambia el nombre por “joint_XX”, donde “XX” es el numero correspondiente del punto detectado por el modelo de deep learning. Como ejemplo se muestra la Figura 38. La enumeración de los puntos de los modelos de deep learning se muestran en la Figura 22 y 23.
- 3) Al modelo se le asigna un script específico que tiene como tareas lo siguiente:
 - Identificar todas las articulaciones y sobre estas crear nuevas articulaciones padre colocadas en la misma posición que las ya existentes pero con una orientación diferente. La orientación debe coincidir con la orientación del sistema de coordenadas del punto 3D correspondiente, así como se definió en

la aplicación de Python y como se explica en el anexo 4. Cada articulación padre tendrá por nombre “joint_father_XX”, donde “XX” corresponde con el número de articulación correspondiente. Como ejemplo la Figura 39. Es importante mencionar que tal cual se trabaja con las orientaciones de un sistema de coordenadas en la aplicación de Python también se trabaja en Unity. Es como trabajar con las orientaciones en Unity como si se estuvieran trabajando en la aplicación de Python. Esto se explica mejor en la Figura 40.

- Mover al modelo según los datos recibidos por parte del servidor. Para esto se hace uso de los ángulos de Euler de cada punto 3D para orientar las nuevas articulaciones que se crearon, tomando en cuenta lo siguiente:
 - El orden de las rotaciones sobre los ejes en la aplicación de Python es Z, Y y X, y para Unity es Y, Z y X (para que sean equivalentes).
 - Las orientaciones positivas en Unity son en sentido de las manecillas del reloj (contrario a lo que se definió en el anexo 4).

Al iniciar el juego el modelo hace uso de los datos recibidos para moverse.

Cabe destacar que la interacción con los modelos 3D no es muy rápida y fluida. Esto mayormente se debe a la gran cantidad de procesamiento que conlleva tener ejecutando la aplicación de Python en donde se ejecutan los modelos de deep learning y se llevan a cabo muchos cálculos en cada instante de tiempo para calcular las estimaciones de puntos 3D. Es razonable que todo esto no sea muy rápido, pero podría mejorar con un equipo de cómputo con mejores características de procesamiento.

La interacción con los modelos 3D se muestra en la Figura 41 y 42.

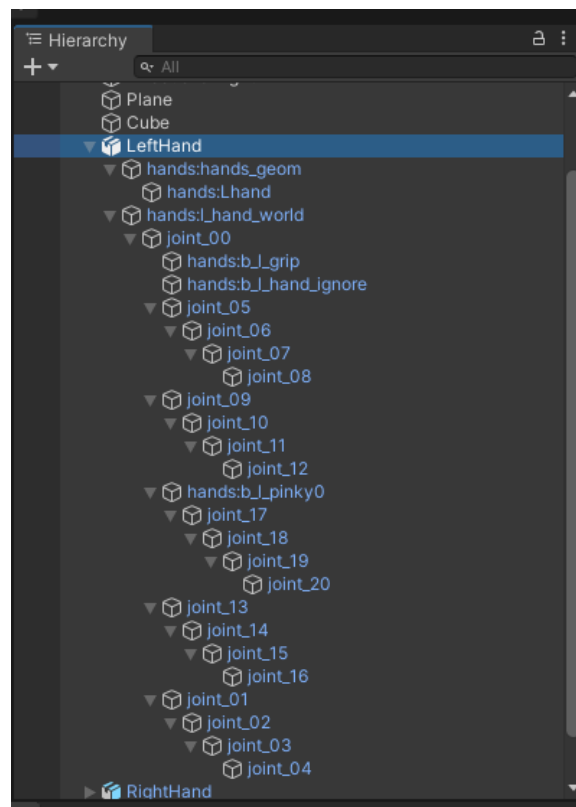


Figura 38. Jerarquía de las articulaciones del modelo 3D de la mano izquierda.

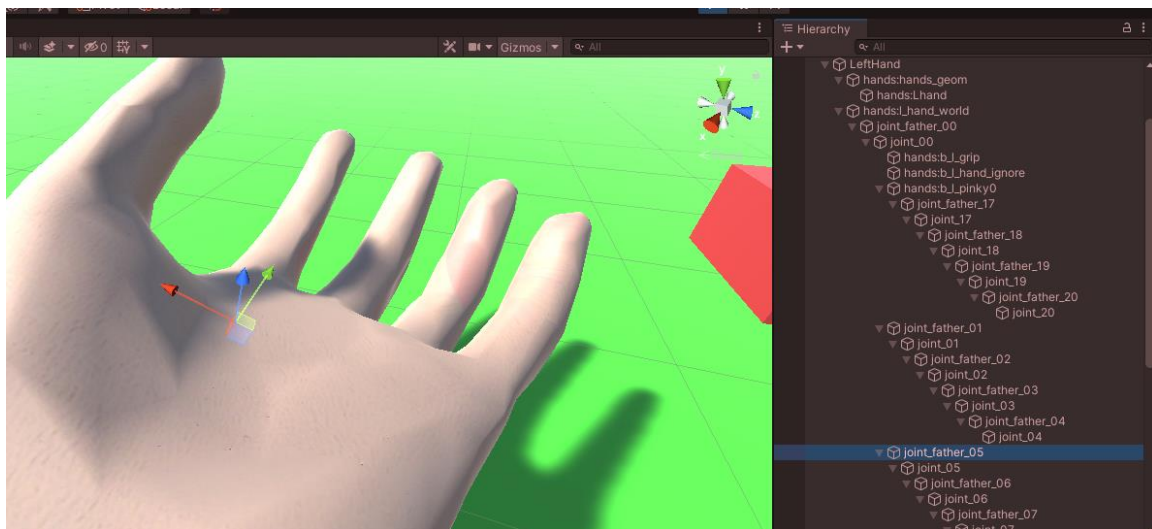


Figura 39. Nuevas articulaciones para el modelo 3D de la mano izquierda.

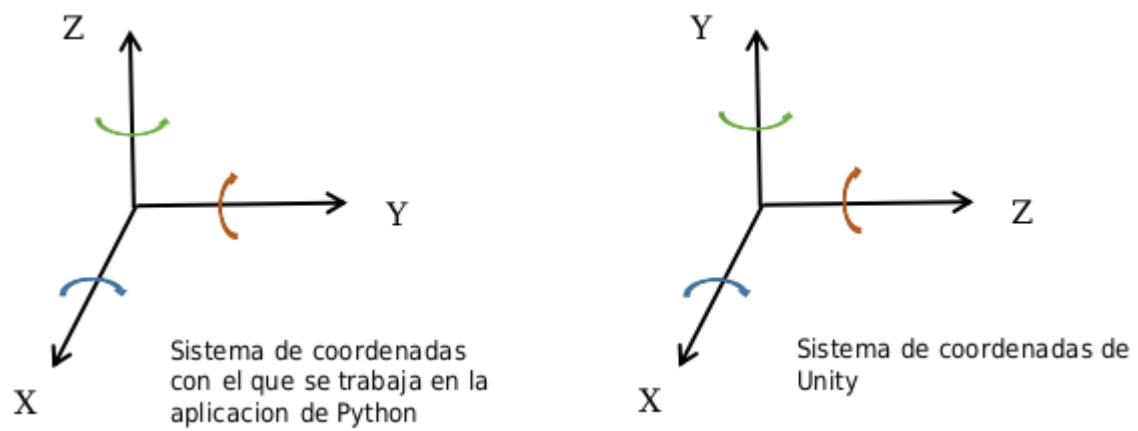


Figura 40. Equivalencia de orientaciones entre los sistemas de coordenadas de la aplicación de Python y la aplicación de Unity.

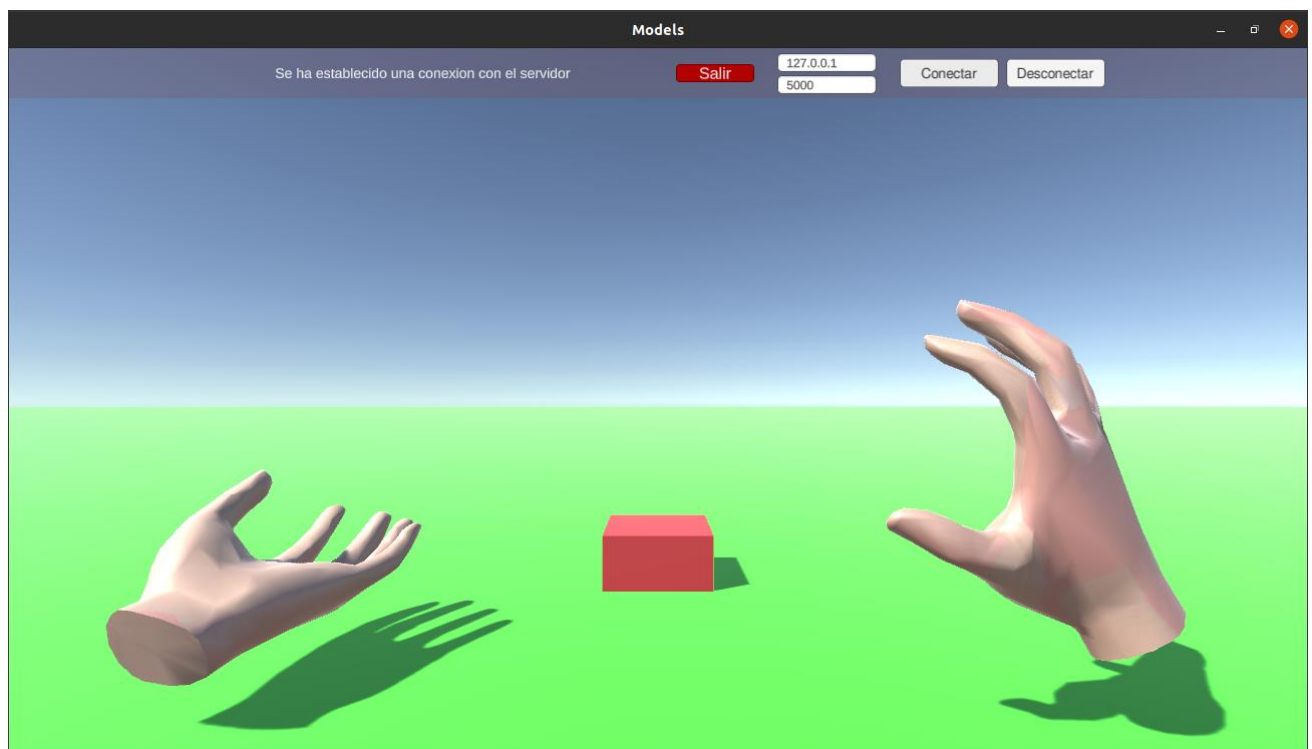


Figura 41. Interacción con los modelos 3D desde el ejecutable de la aplicación de Unity.

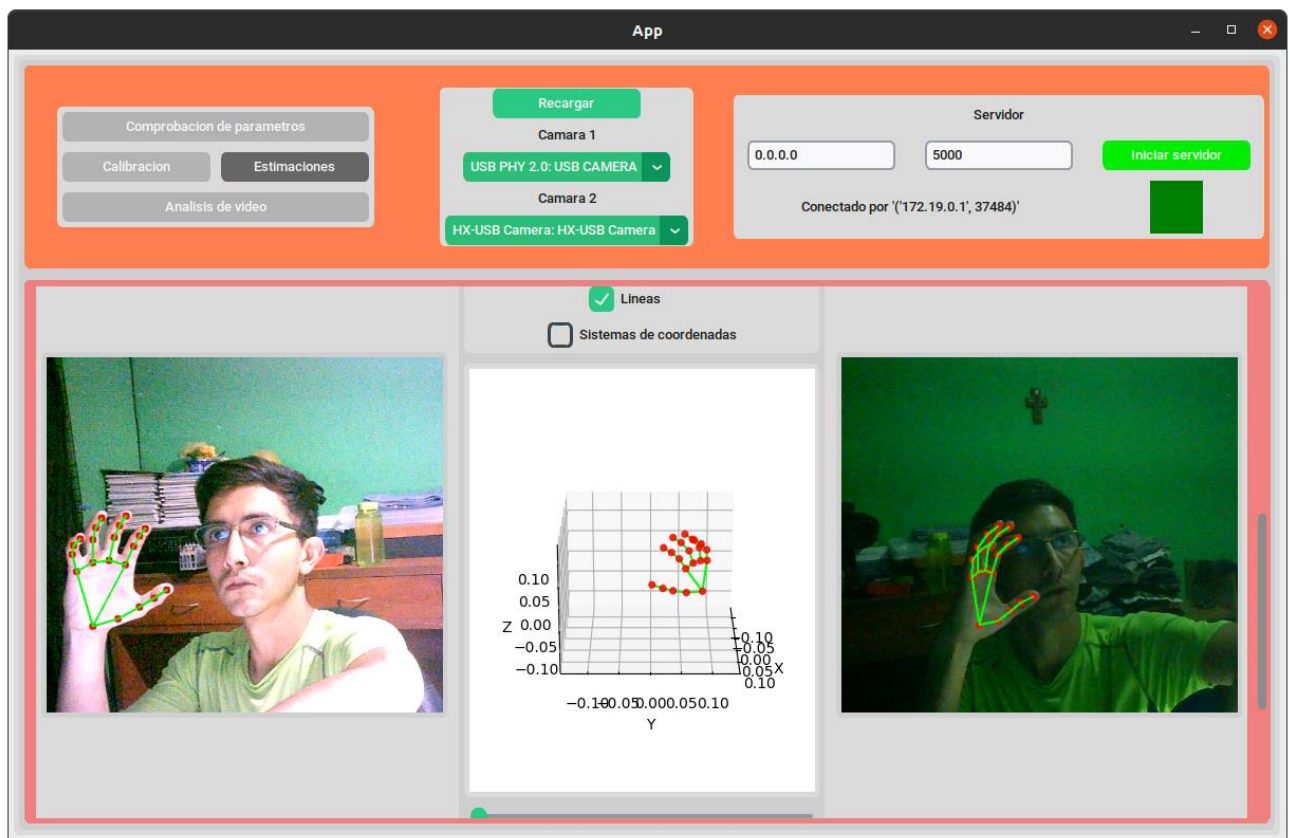


Figura 42. Interacción con los modelos 3D desde la aplicación de Python.

Cuando no se reciben datos para algún modelo 3D, éste se mantiene en la última posición que adoptó, o en su posición inicial en caso de que no haya recibido datos desde un inicio.

Los datos que se reciben pueden ser de diferentes modelos de deep learning y en cualquier orden. El programa en Unity se encarga de separar los datos para los diferentes modelos y de asociarlos con los modelos correspondientes, lo cual da orden y control sobre lo que se recibe, permitiendo extender a más modelos de deep learning y dando la posibilidad de poder utilizar diferentes modelos en la misma escena en Unity.

7. Conclusiones

La aplicación de la visión estéreo para tareas en la terapia física es muy practica y de muy bajo costo, resultando en una herramienta muy útil y poderosa para los fisioterapeutas, la cual implementada de una manera correcta puede ayudar mucho en ciertas tareas relacionadas con el proceso de rehabilitación de los pacientes.

En este trabajo solo se presentó una idea completa del uso de la visión estéreo en el análisis del movimiento corporal de una persona y se presentó un enfoque no terminado de cómo podría usarse para interactuar con juegos 3D creados en Unity para la rehabilitación física. Este último podría ser un trabajo por aparte para quien es experto desarrollando juegos 3D en Unity.

Como cualquier trabajo realizado siempre existen cosas en que mejorar, de las que se destacan una mejor distribución de las opciones en la interfaz de usuario del sistema, ya que el objetivo principal era que cupieran la mayor cantidad de opciones en pantalla, sin embargo, en pantallas pequeñas puede ser algo tediosa la interfaz de usuario. La aplicación de Unity es otra de las cosas que se pueden mejorar incluyendo una escena más completa con juegos interactivos 3D controlados por el movimiento corporal enfocados a la rehabilitación física.

Existe un problema en la aplicación de Python y es con respecto al guardado de los videos que se graban en el apartado de análisis del movimiento corporal. En ocasiones, al momento de detener las grabaciones para guardar los videos, se cierra en automático la aplicación. Es algo que no siempre sucede. El problema está relacionado con la clase *VideoWriter* de la librería OpenCV [4], en específico con los métodos *write* y *release*. El método *write* escribe cada frame y el método *release* termina y guarda la grabación. Al momento de escribir cada frame se espera que sea a una tasa constante, de no cumplirse se genera un error que termina el proceso actual, lo cual no da posibilidades de capturar el error para poder manejarlo. A pesar

del cierre de la aplicación, en la mayoría de los casos, los videos que se graban si se terminan guardando, por lo que se pueden consultar volviendo a ejecutar la aplicación de Python. Uno de los motivos por los cuales los frames no se escriben con una tasa constante es debido a que en la aplicación de Python existen muchos procesos que se ejecutan en paralelo, existen hilos, y realmente asegurar una tasa constante es difícil. Algunas de las alternativas para resolver este problema, como trabajo futuro, es encontrar una manera de poder garantizar una escritura de los frames a una tasa constante.

Uno de los factores muy importantes en el buen desempeño del sistema es el poder computacional del equipo en donde se utilice. Las pruebas que se realizaron sacando el máximo uso del sistema fueron en un equipo de cómputo con procesador Intel i3 8th Gen y con 20GB de RAM, en donde los resultados fueron buenos pero con algo de latencia, esto debido a la gran cantidad de procesamiento que se debe realizar en cada instante.

Trabajos futuros:

- Evaluar todas las posibles implementaciones de la visión estéreo en la terapia física para su implementación y mejora constante, con el objetivo de mejorar y ayudar en el proceso de rehabilitación de los pacientes
- Incursionar en otras áreas de las posibles aplicaciones de la visión estéreo, además de la salud, como lo puede ser la robótica (interacción humano-maquina), el entretenimiento (captura de movimiento, videojuegos), entre otros.

Bibliografía

- [1] Liu, Y., Qiu, C., & Zhang, Z. (2024). Deep learning for 3D human pose estimation and mesh recovery: A survey. *Neurocomputing*, 596, 128049. <https://doi.org/10.1016/j.neucom.2024.128049>.
- [2] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C., & Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2006.10214>.
- [3] Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-device Real-time Body Pose tracking. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2006.10204>.
- [4] OpenCV. (2024, 23 julio). OpenCV - Open Computer Vision Library. <https://opencv.org/>.
- [5] Charmant, J. (s. f.). Kinovea. <https://www.kinovea.org/>.
- [6] Prince, S. J. D. (2012). *Computer Vision: Models, Learning, and Inference*. Cambridge University Press (pp 297 a 304).
- [7] Colaboradores de Wikipedia. (2024, 14 abril). Cámara estenopeica. Wikipedia, la Enciclopedia Libre. https://es.wikipedia.org/w/index.php?title=C%C3%A1mara_estenopeica&oldid=159432332.
- [8] Puebla, J. (2018). Sistema de detección y caracterización de lanzamientos de un pitcher de béisbol utilizando una cámara PlayStation Eye. Unpublished. <https://doi.org/10.13140/RG.2.2.12213.01763>.
- [9] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2.a ed.) [PDF]. Springer. <https://szeliski.org/Book/> (pp 691 a 692).
- [10] Zhang, Y. (2023). 3-D Computer Vision. <https://doi.org/10.1007/978-981-19-7580-6> (pp 37 a 49).
- [11] Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions On Pattern Analysis And Machine Intelligence*, 22(11), 1330-1334. <https://doi.org/10.1109/34.888718>.
- [12] Trefethen, L. N., & Bau, D., III. (1997). *Numerical linear algebra*. SIAM (pp 172 a 175).
- [13] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C., Yong, M. G., Lee, J., Chang, W., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1906.08172>.
- [14] Google-Ai-Edge. (s. f.). mediapipe/docs/solutions/hands.md at master · google-ai-edge/mediapipe. GitHub. <https://github.com/google-ai-edge/mediapipe/blob/master/docs/solutions/hands.md>.
- [15] Google-Ai-Edge. (s. f.-b). mediapipe/docs/solutions/pose.md at master · google-ai-edge/mediapipe. GitHub. <https://github.com/google-ai-edge/mediapipe/blob/master/docs/solutions/pose.md>.
- [16] Blender Foundation. (s. f.). blender.org - Home of the Blender project - Free and Open 3D Creation Software. [blender.org. https://www.blender.org/](https://www.blender.org/).

Anexos

1. Solución ecuación $Ax=b$

Para el siguiente problema por lo general no existe tal solución (x) que haga cumplir exactamente la ecuación:

$$Ax = b$$

El problema anterior se suele plantear de la siguiente manera:

$$e = Ax - b$$

$$f(x) = |e|^2 = e^T e = (Ax - b)^T (Ax - b)$$

Lo que se busca es minimizar la función $f(x)$, es decir, la norma al cuadrado del vector de error. Lo primero que se hara sera determinar su derivada. Una de las formas en las que se puede calcular la derivada de una función es mediante el concepto de diferencial de una función, la cual es un concepto muy poderoso en el cálculo diferencial. El diferencial de una función expresa lo siguiente:

$$df(x) = f(x + dx) - f(x) = \frac{df(x)}{dx} dx$$

Es importante mencionar que los términos de orden superior de diferenciales no se toman en cuenta, ya que se consideran insignificantes. Por ejemplo:

- El cuadrado de un diferencial es cero ($dx^2 = 0$)
- Un diferencial por otro diferencial es cero ($dx dy = 0$)

Para el $df(e)$ se tiene lo siguiente:

$$df(e) = f(e + de) - f(e) = \frac{df(e)}{de} de$$

$$\begin{aligned} df(e) &= (e + de)^T(e + de) - e^T e = (e^T + de^T)(e + de) - e^T e \\ &= e^T e + e^T de + de^T e + de^T de - e^T e = e^T de + de^T e \\ &= e^T de + e^T de = 2e^T de \end{aligned}$$

Para el $de(x)$ se tiene lo siguiente:

$$de(x) = e(x + dx) - e(x) = \frac{de(x)}{dx} dx$$

$$\begin{aligned} de(x) &= (A(x + dx) - b) - (Ax - b) = (Ax + Adx - b) - (Ax - b) \\ &= Ax + Adx - b - Ax + b = Adx \end{aligned}$$

Por tanto, usando los resultados anteriores se tiene lo siguiente:

$$df(x) = 2e^T de = 2e^T (Adx) = 2e^T Adx = 2(Ax - b)^T Adx$$

Por tanto:

$$\frac{df(x)}{dx} = 2(Ax - b)^T A$$

Igualando a cero la derivada y manipulando la expresión se llega al siguiente resultado:

$$x = (A^T A)^{-1} A^T b$$

La ecuación anterior es la solución al problema planteado.

2. Solución ecuación $Ax=0$

Para el siguiente problema la solución trivial es $x = 0$, pero no se esta interesado en esa solución.

$$Ax = 0$$

El problema anterior se suele plantear de la siguiente manera:

$$f(x) = |Ax|^2 = (Ax)^T(Ax) = (x^T A^T)(Ax) = x^T A^T A x$$

Sujeta a: $|x| = 1$

Se tiene un problema de optimización con restricciones de igualdad. Para resolverlo lo que se hace es definir la función lagrangiana del problema, quedando de la siguiente manera:

$$L(x, \lambda) = f(x) - \lambda(x^T x - 1) = x^T A^T A x - \lambda x^T x + \lambda$$

Al resolver para la función lagrangiana también se habrá resuelto para el problema principal.

Por tanto, lo que se busca es minimizar la función $L(x)$. Lo primero que se hará será determinar su derivada. Una de las formas en las que se puede calcular la derivada de una función es mediante el concepto de diferencial de una función, la cual es un concepto muy poderoso en el cálculo diferencial. El diferencial de una función expresa lo siguiente:

$$df(x) = f(x + dx) - f(x) = \frac{df(x)}{dx} dx$$

Es importante mencionar que los términos de orden superior de diferenciales no se toman en cuenta, ya que se consideran insignificantes. Por ejemplo:

- El cuadrado de un diferencial es cero ($dx^2 = 0$)
- Un diferencial por otro diferencial es cero ($dx dy = 0$)

Para el $dL(x)$ se tiene lo siguiente:

$$dL(x) = L(x + dx) - L(x) = \frac{dL(x)}{dx} dx$$

$$dL(x) = (2x^T A^T A - 2\lambda x^T) dx$$

Para el $dL(\lambda)$ se tiene lo siguiente:

$$dL(\lambda) = L(\lambda + d\lambda) - L(\lambda) = \frac{dL(\lambda)}{d\lambda} d\lambda$$

$$dL(\lambda) = (1 - x^T x) d\lambda$$

Las derivadas correspondientes son las siguientes:

$\frac{dL(x)}{dx} = 2x^T A^T A - 2\lambda x^T$
$\frac{dL(\lambda)}{d\lambda} = 1 - x^T x$

Igualando a cero las derivadas y manipulando las expresiones se llega a lo siguiente:

$\frac{dL(\lambda)}{d\lambda} = 0$ $x^T x = 1$ $ x = 1$
$\frac{dL(x)}{dx} = 0$ $A^T A x = \lambda x$

La solución que se busca satisface la ecuación $A^T A x = \lambda x$. Tal ecuación define un problema de valores propios. Por tanto, se tienen un conjunto de soluciones que satisfacen dicho problema.

La pregunta es, ¿cuál es la mejor solución que minimiza a la función $L(x)$?

Sustituyendo $A^T A x = \lambda x$ en la función lagrangiana se tiene lo siguiente:

$L(x, \lambda) = f(x) - \lambda(x^T x - 1) = x^T A^T A x - \lambda x^T x + \lambda = x^T (\lambda x) - \lambda x^T x + \lambda$ $= \lambda x^T x - \lambda x^T x + \lambda = \lambda$

La función lagrangiana va ser mínima si λ es mínima también. Por tanto, la solución al problema es el vector propio con el valor propio asociado más pequeño del siguiente problema:

$A^T A x = \lambda x$

3. Optimizador Adam

Para problemas de optimización, siempre es requerido algún algoritmo de optimización que ayude a encontrar los parámetros óptimos para una función objetivo.

Los optimizadores trabajan en el espacio de entrada de la función objetivo.

Muchos optimizadores hacen uso del filtro EWMA (Exponentially Weighted Moving Average), el cual ayuda a filtrar mediciones. Este filtro da más importancia a las mediciones actuales que a las mediciones anteriores (no de manera lineal, si no de manera exponencial).

Su ecuación es la siguiente:

$v_t = \beta v_{t-1} + (1 - \beta)m_t$
Donde: ● $0 < \beta < 1$ ● m_t: Medicion actual ● v_{t-1}: Prediccion anterior

De los optimizadores más utilizados se encuentra Adam, el cual significa “estimación de momento adaptativo”. Es un algoritmo que calcula las tasas de aprendizaje adaptativo para cada parámetro.

El algoritmo de Adam es el siguiente:

Repetir hasta converger: $v_{(g)t} = \beta_1 v_{(g)t-1} + (1 - \beta_1) \nabla f(\theta_t)$ $v_{(gs)t} = \beta_2 v_{(gs)t-1} + (1 - \beta_2) \nabla f(\theta_t)^2$ $\overline{v_{(g)t}} = \frac{v_{(g)t}}{1 - \beta_1^t}$ $\overline{v_{(gs)t}} = \frac{v_{(gs)t}}{1 - \beta_2^t}$ $\theta_t = \theta_{t-1} - \alpha \frac{1}{\sqrt{\overline{v_{(gs)t}} + \epsilon}} \overline{v_{(g)t}}$
Donde: ● θ_t: Parametros de la funcion objetivo en un instante de tiempo t ● α: Tasa de aprendizaje ● ϵ: 1e-5 (para evitar la división entre cero) ● β_1: 0.9

- $\beta_2: 0.999$

$\overline{v_{(g)t}}$ y $\overline{v_{(gs)t}}$ se llevan a cabo para la corrección de sesgo de $v_{(g)t}$ y $v_{(gs)t}$, respectivamente.

Se sabe que el cambio de los parámetros es inversamente proporcional a su derivada.

4. Ángulos de Euler a partir de estimaciones de puntos 3D

Consideraciones antes de continuar:

- El sistema de coordenadas de referencia que se usa para el análisis en este anexo es el mismo que el de la Figura 25.
- Para las matrices de rotación, las rotaciones positivas son en sentido contrario a las manecillas del reloj.

Los ángulos de Euler corresponden a los 3 ángulos de una matriz de rotación dada por la siguiente ecuación:

$$R(\psi, \theta, \phi) = R_Z(\phi)R_Y(\theta)R_X(\psi) = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta \cos \phi & \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi & \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi \\ \cos \theta \sin \phi & \sin \psi \sin \theta \sin \phi + \cos \psi \cos \phi & \cos \psi \sin \theta \sin \phi - \sin \psi \cos \phi \\ -\sin \theta & \sin \psi \cos \theta & \cos \psi \cos \theta \end{bmatrix}$$

Donde el orden de las rotaciones es el siguiente:

- 1) Eje Z
- 2) Eje Y
- 3) Eje X

Donde:

$$R_X(\psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \psi & -\sin \psi \\ 0 & \sin \psi & \cos \psi \end{bmatrix}$$

$R_Y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$
$R_Z(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Ahora bien, es posible calcular los ángulos de Euler a partir de una matriz de rotación en R^3 . Esto se logra mediante un algoritmo que toma la matriz R y devuelve los ángulos ψ , θ y ϕ (en ese orden).

Antes de mostrar el algoritmo es importante conocer lo siguiente, ya que siempre que se usan las funciones trigonométricas inversas para encontrar ángulos es recomendable conocer su dominio y su rango:

$y = \sin^{-1} x$
Dominio: $[-1, 1]$ Rango: $[-\frac{\pi}{2}, \frac{\pi}{2}]$
$y = \cos^{-1} x$
Dominio: $[-1, 1]$ Rango: $[0, \pi]$
$y = \tan^{-1} x$
Dominio: $[-\infty, \infty]$ Rango: $[-\frac{\pi}{2}, \frac{\pi}{2}]$
$\text{atan2}(y, x)$
Es un algoritmo que la mayoría de lenguajes de programación implementa. Mejora los resultados de arctan debido a que tiene rango completo y devuelve el ángulo correcto. Se usa este algoritmo en lugar de $y = \tan^{-1} x$. Dominio: $[-\infty, \infty]$

Rango: $(-\pi, \pi]$

Consideraciones:

- No existe una solución cerrada para este problema, es decir, se tienen varios resultados que son válidos.
- Con las ecuaciones que resultan de la matriz de rotación se busca encontrar los valores de los ángulos ψ , θ y ϕ . Esto involucra seleccionar ecuaciones, despejar variables, utilizar ecuaciones en otras ecuaciones, etc.
- Todo ángulo despejado por $\arctan$ se busca que sea de la forma atan2 .
- Se debe tener cuidado de no eliminar variables cuando se pueda (ya que proporcionan información valiosa) al tratar con las ecuaciones.

Algoritmo para encontrar los ángulos de Euler:

- Si $R_{31} \neq -1$ y $R_{31} \neq 1$ (caso cuando $\cos \theta \neq 0$)

$$\theta_1 = \sin^{-1}(-R_{31})$$

$$\theta_2 = \pi - \theta_1$$

$$\psi_1 = \text{atan2}\left(\frac{R_{32}}{\cos \theta_1}, \frac{R_{33}}{\cos \theta_1}\right)$$

$$\psi_2 = \text{atan2}\left(\frac{R_{32}}{\cos \theta_2}, \frac{R_{33}}{\cos \theta_2}\right)$$

$$\phi_1 = \text{atan2}\left(\frac{R_{21}}{\cos \theta_1}, \frac{R_{11}}{\cos \theta_1}\right)$$

$$\phi_2 = \text{atan2}\left(\frac{R_{21}}{\cos \theta_2}, \frac{R_{11}}{\cos \theta_2}\right)$$
- De lo contrario (caso cuando $\cos \theta = 0$)
 - Si $R_{31} = -1$ (caso cuando $-\sin \theta = -1 \rightarrow \theta = \frac{\pi}{2}$)

$$\theta = \frac{\pi}{2}$$

$$\phi = \text{Arbitrario (puede ser 0)}$$

$$\psi = \text{atan2}(R_{12}, R_{13}) + \phi$$

- De lo contrario (caso cuando $-\sin \theta = 1 \rightarrow \theta = -\frac{\pi}{2}$)

$$\theta = -\frac{\pi}{2}$$

$$\phi = \text{Arbitrario (puede ser 0)}$$

$$\psi = \text{atan2}(-R_{12}, -R_{13}) - \phi$$

El algoritmo anterior obtiene los ángulos de Euler en radianes y en el siguiente orden: ψ , θ y ϕ . Si se necesitan en grados solo se tiene que hacer una conversión de radianes a grados.

Ahora, en base a las estimaciones de puntos 3D, los cuales se encuentran expresadas en el sistema de coordenadas de la Fig. 25, es posible definir un sistema de coordenadas en cada punto 3D construyendo los vectores base de dicho sistema en función de otros puntos 3D, siempre considerando que estos vectores deben formar un conjunto ortonormal de vectores. Al construir estos sistemas de coordenadas lo que se tendrá será una matriz de rotación de dicho sistema de coordenadas con respecto al sistema de coordenadas de la Fig. 25. Con esta información es posible encontrar los ángulos de Euler para cada matriz de rotación de cada punto 3D. Estos ángulos de Euler son los que también se envían a Unity (pero en grados).

Para cada modelo se elige como se van a construir los vectores base de cada sistema de coordenadas, ya que dependen de los puntos 3D del propio modelo. En general, la construcción de estos vectores base sigue la siguiente estrategia apoyada en la Tabla 1:

$$\begin{aligned} v_{m,n} &= v_n - v_m \\ v_{p,q} &= v_q - v_p \\ v &= v_{p,q} \times v_{m,n} \\ u^{(Z)} &= \frac{v_{m,n}}{|v_{m,n}|} \\ u^{(Y)} &= \frac{v}{|v|} \\ u^{(X)} &= u^{(Y)} \times u^{(Z)} \\ \text{Vectores base: } &[u^{(X)} \quad u^{(Y)} \quad u^{(Z)}] \end{aligned}$$

Donde:

- v_k : Representación 3D del punto k del modelo de deep learning
- $u^{(D)}$: Vector unitario en direccion del eje D

MediaPipe Hands					MediaPipe Pose				
Punto	m	n	p	q	Punto	m	n	p	q
0	0	9	0	17	0	0	9	0	10
1	1	2	0	17	1	0	9	0	10
2	2	3	0	17	2	0	9	0	10
3	3	4	0	17	3	0	9	0	10
4	3	4	0	17	4	0	9	0	10
5	5	6	5	9	5	0	9	0	10
6	6	7	5	9	6	0	9	0	10
7	7	8	5	9	7	0	9	0	10
8	7	8	5	9	8	0	9	0	10
9	9	10	5	9	9	0	9	0	10
10	10	11	5	9	10	0	9	0	10
11	11	12	5	9	11	11	13	11	23
12	11	12	5	9	12	12	14	12	24
13	13	14	5	9	13	13	15	13	24
14	14	15	5	9	14	14	16	14	23
15	15	16	5	9	15	15	19	15	17
16	15	16	5	9	16	16	20	16	18
17	17	18	5	9	17	17	15	17	19
18	18	19	5	9	18	18	16	18	20
19	19	20	5	9	19	19	17	19	15
20	19	20	5	9	20	20	18	20	16
					21	21	15	21	17
					22	22	16	22	18
					23	23	25	23	24
					24	24	26	24	23
					25	25	27	25	26
					26	26	28	26	25
					27	27	31	27	29
					28	28	32	28	30

	29	29	31	29	27
	30	30	32	30	28
	31	31	29	31	27
	32	32	30	32	28

Tabla 1. Selección de puntos para la elaboración de sistemas de coordenadas para los diferentes modelos de deep learning basado en la estrategia planteada.

La Figura 43 muestra un ejemplo del cálculo de sistemas de coordenadas para el modelo MediaPipe Hands.

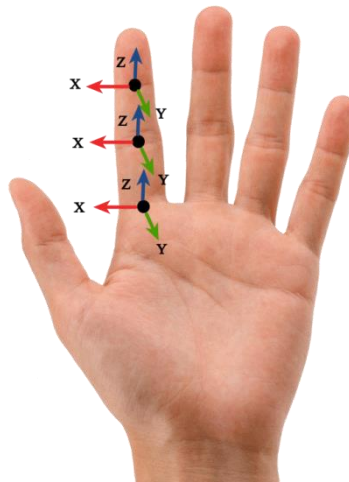


Figura 43. Algunos sistemas de coordenadas de puntos del dedo índice de la mano izquierda.